

דו"ח מכין למעבדה 5

Scalar Pipelined RISC-V CPU Architecture

מעבד RV32IM - מימוש Single-Cycle ומימוש Pipeline בן 5 שלבים

שם הקורס	מעבדת ארכיטקטורת מעבדים מתקדמת ומאיצי חומרה
מספר קורס	361.1.4693
מנחה	ד"ר חנן ריבו
מגישים	אדר שפירא 209580208 יהונתן דדכה 211468582
תאריך הגשה	24.06.2026

1. מטרת המעבדה ורקע תיאורטי

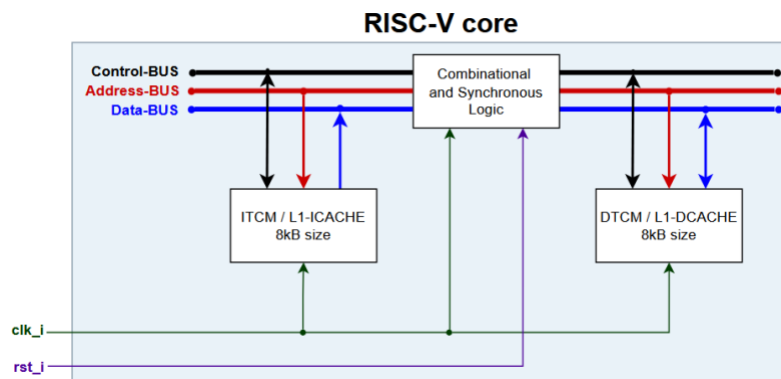
מטרת מעבדה זו היא לתכנן, לסנתז ולנתח מעבד תואם RISC-V, ולהעלות אותו ממימוש single-cycle למימוש pipeline סקלרי (scalar pipelined) בן חמישה שלבים. העבודה התמקדה בארבעה נושאים:

- תכנון, סינתזה וניתוח של מעבד תואם RISC-V על גבי Intel FPGA.
 - מימוש מיקרו-ארכיטקטורת pipeline הכוללת יחידת forwarding מלאה ויחידת גילוי hazards בגישת combinational check.
 - ההבדל בין CPU ל-MCU: הליבה שתכננו היא מעבד "עירום", בלי היקפיים, בלי פסיקות ובלי בקרי קלט/פלט. יש בה מסלול נתונים, יחידת בקרה ושני זיכרונות צמודים בלבד.
 - מבנה הזיכרון ב-FPGA: את ה-ITCM ואת ה-DTCM מימשנו באמצעות בלוקי M9K המשובצים ברכיב, דרך megafunction מסוג altsyncram.
- ארכיטקטורת RISC-V היא ארכיטקטורת Harvard: זיכרון ההוראות וזיכרון הנתונים מופרדים לחלוטין, ולכל אחד מהם אפיק משלו. ההפרדה מעלה את קצב התפוקה (throughput), מפני שאפשר לשלוף הוראה ולגשת לנתון באותו מחזור שעון. היא גם מפשטת את הלוגיקה של שלב ה-MEM ב-pipeline, שכן אין תחרות (structural hazard) בין שליפת הוראה לבין גישה לזיכרון נתונים.

2. הגדרת המשימה וארכיטקטורת המערכת

2.1 ארכיטקטורת המערכת הכוללת

המערכת בנויה סביב ליבת RISC-V יחידה, המחוברת לשני זיכרונות צמודים בגודל 8 kB כל אחד: ITCM (L1-ICACHE) לאחסון הקוד, ו-DTCM (L1-DCACHE) לאחסון הנתונים. הליבה מקבלת שעון יחיד (`clk_i`) ואיפוס (`rst_i`), כאשר האיפוס מחווט ללחצן `KEY0` שעל הכרטיס ומחזיר את ה-PC להוראה הראשונה בתוכנית.



איור 1: ארכיטקטורת המערכת: ליבת RISC-V עם זיכרונות ITCM ו-DTCM צמודים (מתוך הגדרת המשימה)

הן הישות העליונה והן ליבת ה-RISC-V מומשו כתכנ מבני (structural), כלומר כחיווט מפורש של תת-מודולים ולא כתיאור התנהגותי מונוליתי. כך נשמרת התאמה אחד-לאחד בין קוד ה-VHDL לבין תרשים המיקרו-ארכיטקטורה, ובעזרת ה-RTL Viewer אפשר לאתר בקלות כל בלוק שמופיע בתרשים.

2.2 סביבת הפיתוח והכלים

רכיב	ערך	הערה
כרטיס פיתוח	DE2-115	-
רכיב FPGA	EP4CE115F29C7	Cyclone IV E
כלי סינתזה	Quartus Prime 21.1.0 Lite	קלט VHDL-2008
כלי סימולציה	ModelSim	מחזור שעות 100 ns ב-testbench
מודל זהב	RARS	השוואת DTCM.mem מול DTCM.h
מקור שעות על הכרטיס	50 MHz	נכנס ל-clk_i
שעות הליבה MCLK	25 MHz	נגזר ב-PLL: $2/1 \times 50$

טבלה 1: סביבת הפיתוח והכלים ששימשו בעבודה

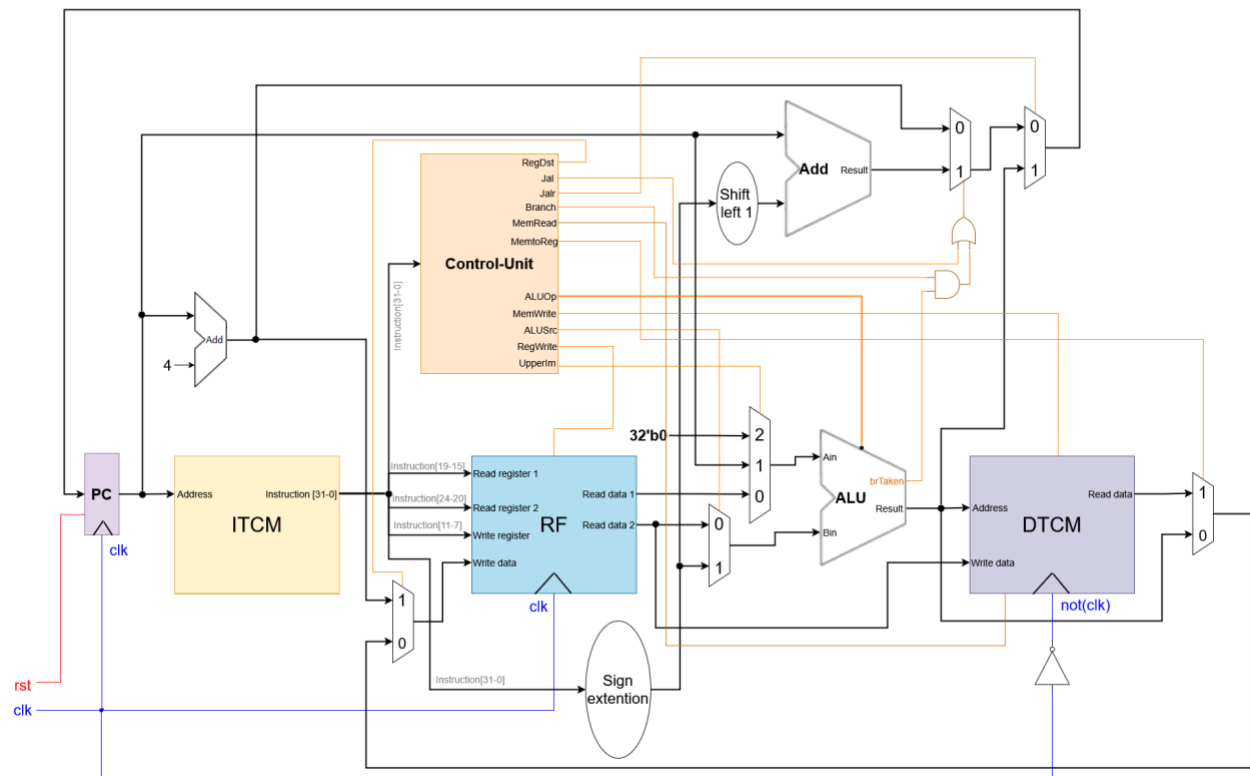
התכן משתמש בקומפילציה מותנית (`cond_compilation_package.vhd`) כדי לעבוד בשתי סביבות מאותו קוד מקור. כאשר `G_MODELSIM = 1` הזיכרונות נטענים מקבצי `hex` וה-PLL אינו מיוצר כלל, והשעות מוזן ישירות מה-testbench. כאשר `G_MODELSIM = 0` נוצר רכיב ALTPLL הגוזר את שעות הליבה מהמתנד שעל הכרטיס. שאר הפרמטרים בחבילה קובעים את גודל ה-TCM (8 KiB), את רוחב ה-PC ואת רזולוציית הכתובות (`G_WORD_GRANULARITY = True`, כלומר כתובת ייחודית לכל מילה).

3. חלק 1 - מיקרו־ארכיטקטורת Single-Cycle RV32IM

3.1 נקודת המוצא: Single-Cycle RV32I

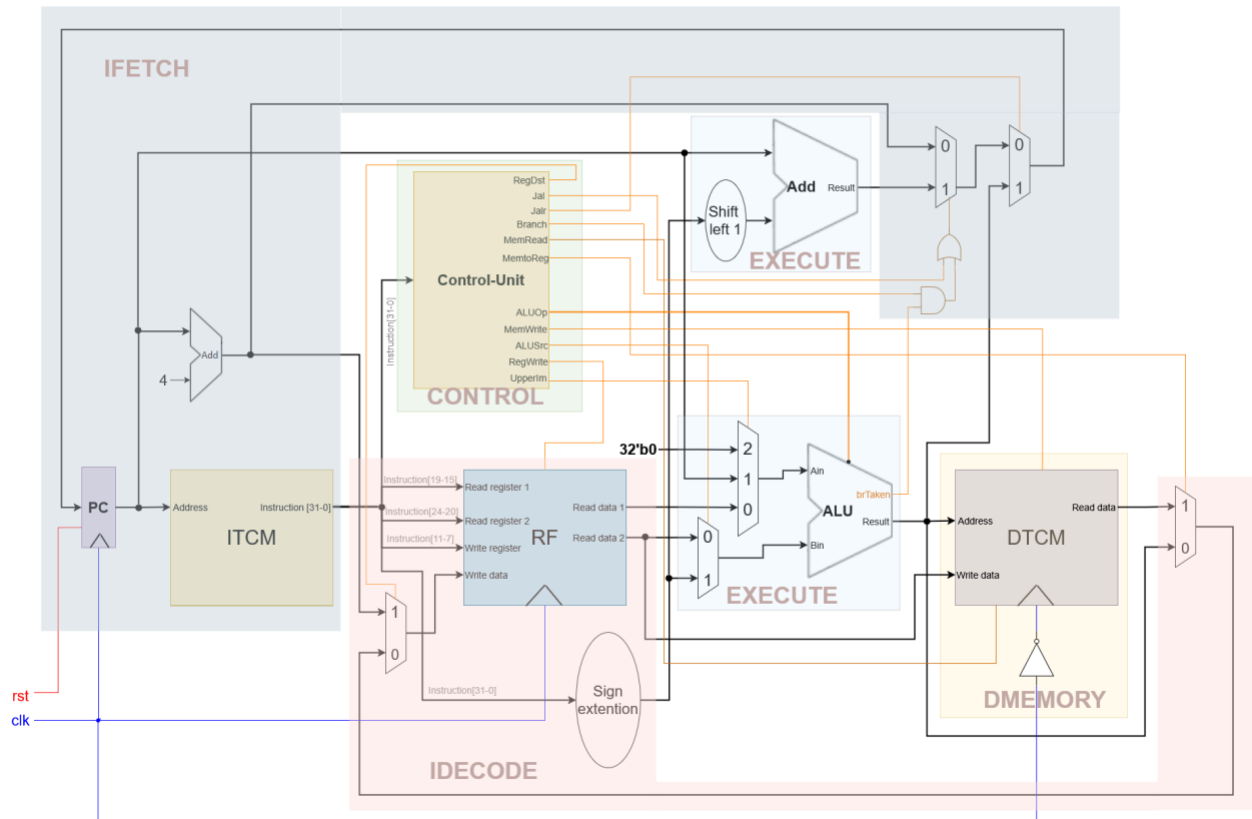
כנקודת מוצא קיבלנו מיקרו־ארכיטקטורת single-cycle התומכת ב-RV32I, ה-ISA הבסיסי הלא־מיוחס. במימוש זה כל הוראה מסיימת את ביצועה במחזור שעון יחיד. ה-PC מספק את כתובת ההוראה, ה-ITCM מחזיר את ההוראה, יחידת הבקרה מפענחת אותה, קובץ האוגרים נקרא, ה-ALU מבצע את החישוב, ה-DTCM נגיש לקריאה או לכתיבה, והתוצאה נכתבת בחזרה לקובץ האוגרים, הכול בתוך אותו מחזור.

RISC-V RV32I Single-Cycle uArchitecture (supports the base Integer unprivileged ISA) ©Hanan Ribo



איור 2: מיקרו־ארכיטקטורת Single-Cycle RV32I: נקודת המוצא של המשימה

המימוש מחולק לחמישה תת־מודולים פונקציונליים המהווים גם את הבסיס לחלוקה לשלבי ה-pipeline בחלק 2: IFETCH, IDCODE, CONTROL, EXECUTE ו-DMEMORY.



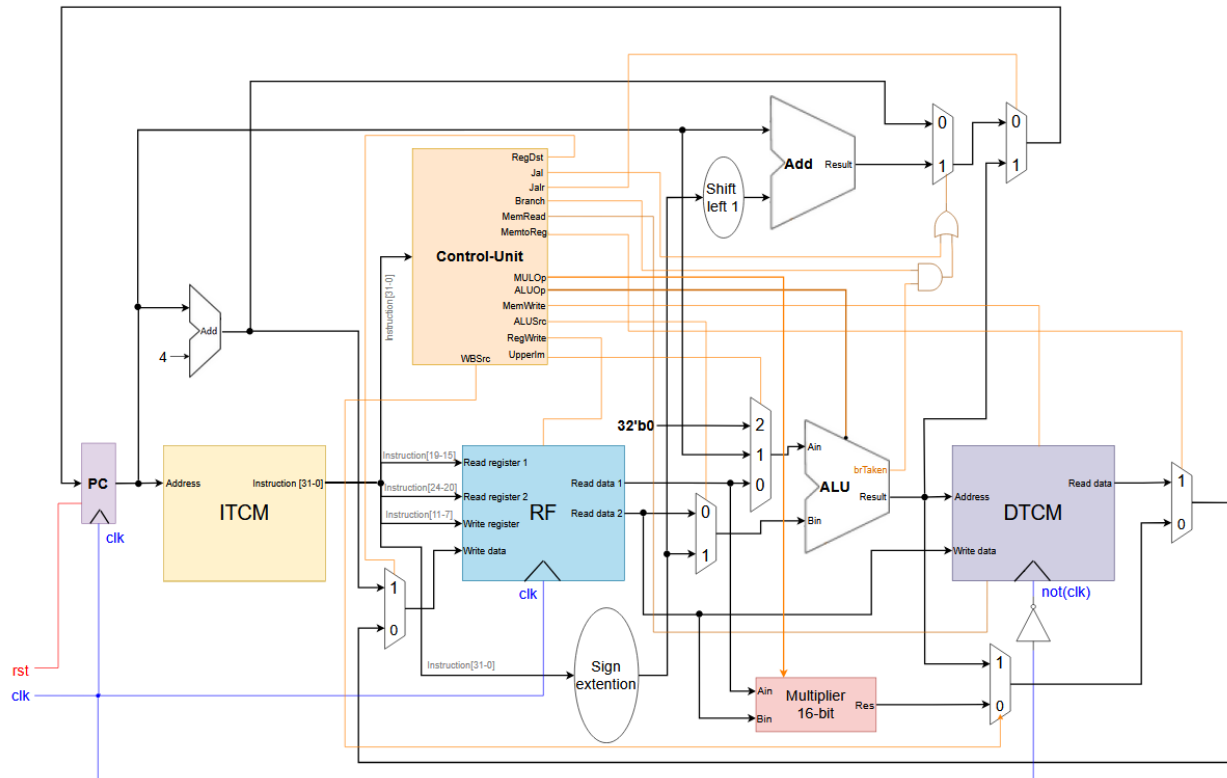
איור 3: חלוקת המיקרו־ארכיטקטורה לתת־מודולים (submodules)

ה-DTCM נכתב על השעון ההפוך ($\text{not}(\text{clk})$). הסיבה: במימוש single-cycle, כתובת הזיכרון היא תוצאת ה-ALU, שמתייצבת רק בסוף המחזור. כתיבה על העלייה הבאה של השעון הייתה מאחרת את הנתון במחזור שלם. הכתיבה על הירידה מקצה חצי מחזור לייצוב הכתובת וחצי מחזור לפעולת הזיכרון עצמה. להחלטה זו יש השלכה ישירה על ניתוח התזמון בסעיף 6.2: המסלול הקריטי של המימוש הוא מסלול חצי מחזור, ולכן $f_{\max}$ נקבע לפי $1/2$ ולא לפי $t_{\text{path}}/1$.

3.2 הרחבת M: מימוש פקודת mul

משימת חלק 1 היא להעלות את המיקרו־ארכיטקטורה מ-RV32I ל-RV32IM. הרחבת M מגדירה פעולות כפל וחילוק (MULDIV), אך דרישת המשימה מסתפקת בתמיכה חלקית: פקודת mul בלבד מתוך קבוצת הכפל (mul, mulhu, mulhsu, mulh).

סמנטיקת הפקודה כפי שהוגדרה במשימה: `mul rd, rs1, rs2` מכפילה את חצאי המילה התחתונים (16 ביט) של האוגרים `rs1` ו-`rs2`, וכותבת תוצאה בת 32 ביט לאוגר `rd`. הפקודה היא מסוג R-type, ולכן כל אותות הבקרה האחרים (MemtoReg, RegWrite וכו') זהים לאלו של פעולת ALU רגילה. ההרחבה מתבטאת אך ורק בקידוד ALUOp חדש ובנתיב נתונים נוסף.



איור 4: מיקרו־ארכיטקטורת Single-Cycle RV32IM (MULDIV partial): הכופל בן 16 הביט משולב לצד ה-ALU, ומרבב חדש בוחר בין שתי התוצאות

3.2.1 אלגוריתם הכפל

האלגוריתם הנדרש מפרק כפל 16×16 לארבעה מכפלים חלקיים בני 8×8 , כך שהסינתזה תמפה אותם ישירות על הכופלים המשובצים (embedded 9-bit multipliers) של ה-FPGA במקום לבנות כופל מלוגיקה. הפירוק מתבצע בשני שלבים:

$$\text{Stage 1 } P0 = A_{\text{low}} \times B_{\text{low}} \cdot P1 = A_{\text{low}} \times B_{\text{high}} \cdot P2 = A_{\text{high}} \times B_{\text{low}} \cdot P3 = A_{\text{high}} \times B_{\text{high}}$$

$$\text{Stage 2 } M = P1 + P2 \cdot \text{RESULT} = P0 + (M \ll 8) + (P3 \ll 16)$$

המימוש שלנו נמצא בקובץ `MUL16.vhd` ומתרגם את המשוואות אחד-לאחד: כל אופרנד מפוצל לשני חצאים בני 8 ביט, ארבעת המכפלים החלקיים מחושבים במקביל, הסכום האמצעי $M = P1 + P2$ מחושב ברוחב 17 ביט (ביט נוסף לנשא), ולבסוף שלושת האיברים מיושרים לרוחב 32 ביט ומחוברים. כדי לוודא שהמיפוי אכן התבצע כמתוכנן בדקנו את דוח ה-Area, והוא מדווח על 4 יחידות **embedded 9-bit multiplier** בשני המימושים (סעיף 6.1), בדיוק כמספר המכפלים החלקיים. כלומר הסינתזה מיפתה את הכפל לחומרה הייעודית ולא ללוגיקה.

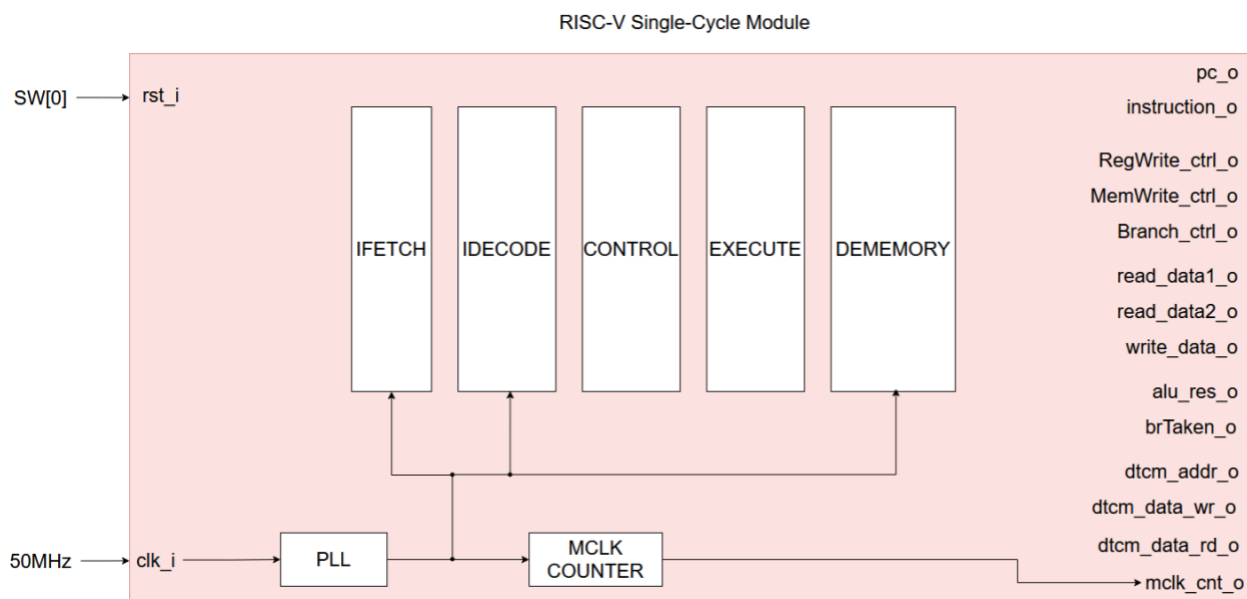
3.2.2 השינויים שנדרשו בקבצי התכנ

קובץ	השינוי שבוצע
const_package.vhd	הוספת תבנית הזיהוי INST_MUL ומסכת הביטים INST_MUL_MASK, והוספת הקידוד ALU_MUL לרשימת פעולות ה-ALU המשותפת ל-CONTROL ול-EXECUTE.
CONTROL.vhd	זיהוי הפקודה לפי INST_MUL_MASK/INST_MUL והנעת ALUOp = ALU_MUL. שאר אותות הבקרה נותרו ללא שינוי, שכן mul היא R-type.
MUL16.vhd	מודול חדש - הכופל $16 \times 16 \rightarrow 32$ על בסיס ארבעה מכפלים חלקיים.
EXECUTE.vhd	יצירת מופע של MUL16 ובחירת פלטו כ-alu_res כאשר ALUOp = ALU_MUL.
aux_package.vhd	הוספת הצהרת הרכיב MUL16 ושינוי שם הליבה מ-RV32I_CORE ל-RV32IM_CORE.

טבלה 2: סיכום השינויים במעבר מ-RV32I ל-RV32IM במימוש ה-single-cycle

3.3 הישות העליונה ותרשים בלוקים

הישות העליונה RV32IM_CORE מחווטת את חמשת תת-המודולים ומוציאה החוצה קבוצת אותות ניפוי המשמשים הן את חלון הגלים ב-ModelSim והן את הוולידציה ב-Signal-Tap על הכרטיס: ה-PC, ההוראה, אותות הבקרה המרכזיים (RegWrite_ctrl_o, MemWrite_ctrl_o, Branch_ctrl_o), נתוני קובץ האוגרים, תוצאת ה-ALU, אפיקי ה-DTCM ומונה מחזורי השעון mclk_cnt_o.



איור 5: הישות העליונה של מימוש ה-Single-Cycle: RV32IM_CORE ואותות הניפוי שלה

במימוש single-cycle כל הוראה נמשכת מחזור אחד בדיוק, ולכן $IPC = 1$ בהגדרה ומונה מחזורי השעון בסוף הריצה שווה למספר ההוראות שבוצעו בפועל. השתמשנו בכך כמודל זהב עצמאי לבדיקת נוסחת ה-IPC של ה-pipeline בסעיף 7: מספר ההוראות שפרשו (retired) שמחושב מהמונים של ה-pipeline חייב להיות שווה למונה המחזוריים של ה-single-cycle באותה אפליקציה.

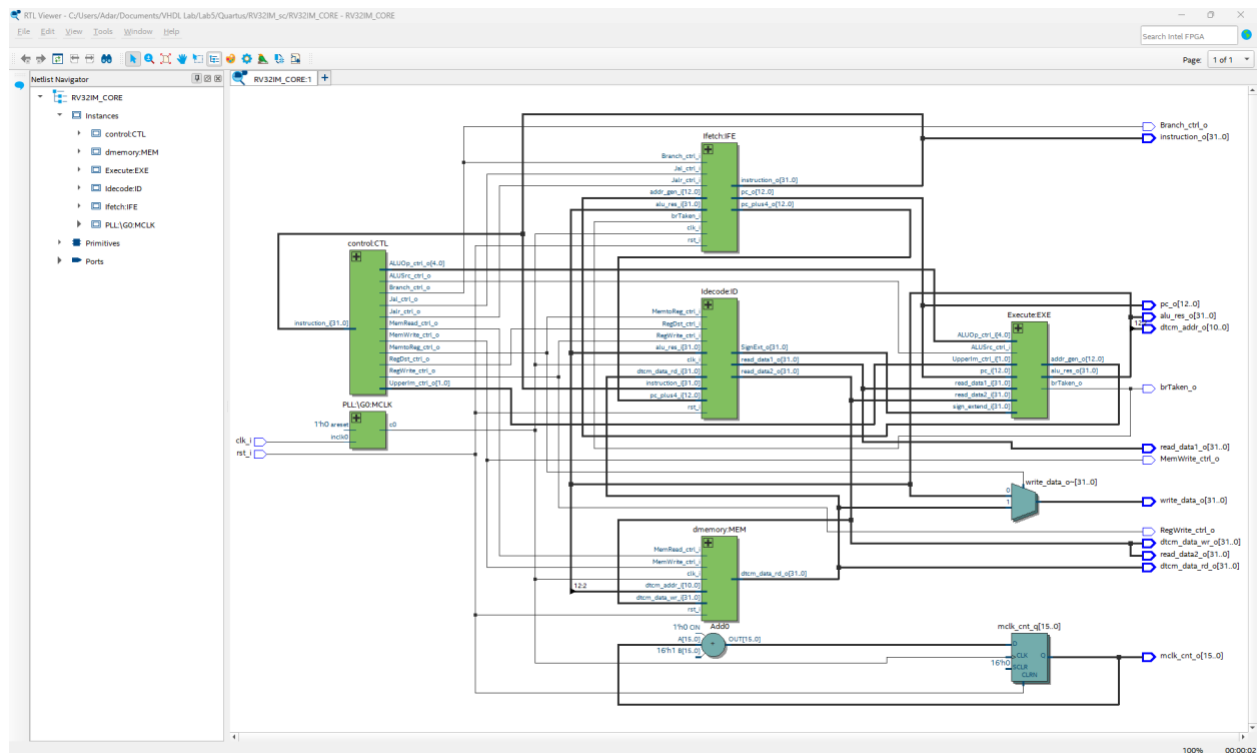
3.4 תיאור קבצי ה-HDL

קובץ	תיאור
cond_compilation_package.vhd	פרמטרי קומפילציה מותנית: מיתוג FPGA/ModelSim (G_MODELSIM), גודל ה-TCM (1–8 KiB), רוחבי PC ו-MA, ויחסי החלוקה והכפל של ה-PLL.
const_package.vhd	תבניות ה-opcode ומסכות ההוראות שבהן משתמש CONTROL, וקידוד ALUOp המשותף ל-CONTROL ול-EXECUTE (כולל ALU_MUL שנוסף עבור הרחבת M).
MUL16.vhd	כופל $16 \times 16 \rightarrow 32$ של חצאי המילה התחתונים של rs2/rs1, הבנוי מארבעה מכפלים חלקיים בני 8 ביט לפי $RESULT = P0 + ((P1+P2) << 8) + (P3 << 16)$, כך ש-Quartus ממפה אותו לכופלים המשובצים.
CONTROL.vhd	מפענח צירופי (combinational decoder); מזהה mul באמצעות $INST_MUL_MASK/INST_MUL$ ומניע ALUOp = ALU_MUL.
EXECUTE.vhd	ה-ALU (חיבור/חיסור, פעולות לוגיות, barrel shifters, השוואות) ולוגיקת ההסתעפות; יוצר מופע של MUL16 ובוחר את תוצאתו כ-alu_res עבור ALU_MUL.
IFETCH.vhd	אוגר ה-PC, מחבר PC+4, מרבב ה-PC הבא (jalr/jal/branch) וה-ITCM (altsyncram).
IDECODE.vhd	קובץ אוגרים 32×32 , חילוץ שדות ההוראה, כל חמש תבניות ה-immediate והרחבת הסימן, ומרבב ה-write-back.
DMEMORY.vhd	ה-DTCM (altsyncram), הנכתב על השעון ההפוך.
aux_package.vhd	הצהרות הרכיבים של כל התכן.
PLL.vhd	ALTPLL הגוזר את שעון הליבה בן 25 MHz מהמתנד בן 50 MHz שעל הכרטיס; נוצר רק כאשר $G_MODELSIM = 0$.
RV32IM_CORE.vhd	הישות העליונה המבנית; מחווטת את חמשת תת-המודולים וחושפת את אותות הניפוי ל-Signal-Tap.

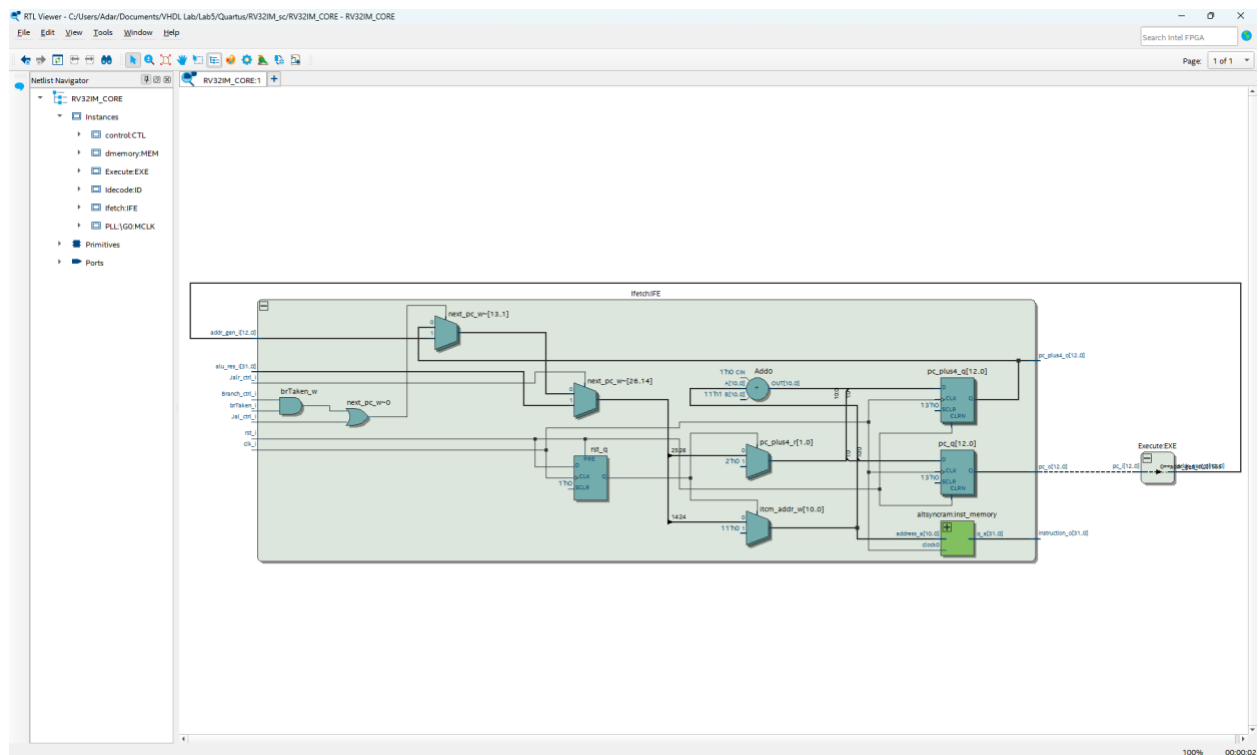
טבלה 3: תיאור קבצי ה-HDL של מימוש ה-Single-Cycle RV32IM (תיקיה DUT/RV32IM_sc)

3.5 תוצאות RTL Viewer

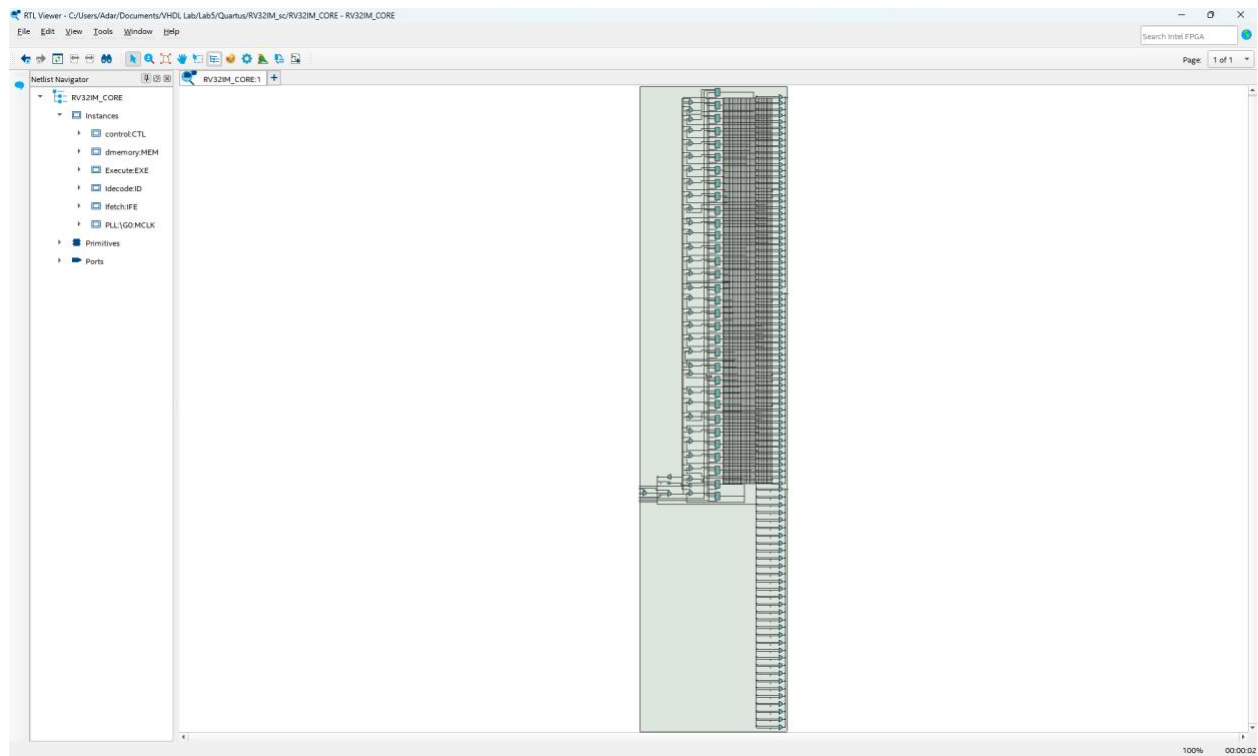
לאחר הסנתזה ב-Quartus בחנו את התכן ב-RTL Viewer כדי לוודא שהמבנה שהתקבל תואם לתרשים המיקרו-ארכיטקטורה. מכיוון שהתכן העליון מבני, הבלוקים בתצוגה מתאימים אחד-לאחד לבלוקים שבאיור 4.



איור 6: הרמה העליונה של RV32M_CORE ב-RTL Viewer: חמשת תת-המודולים ואפיקי החיבור ביניהם



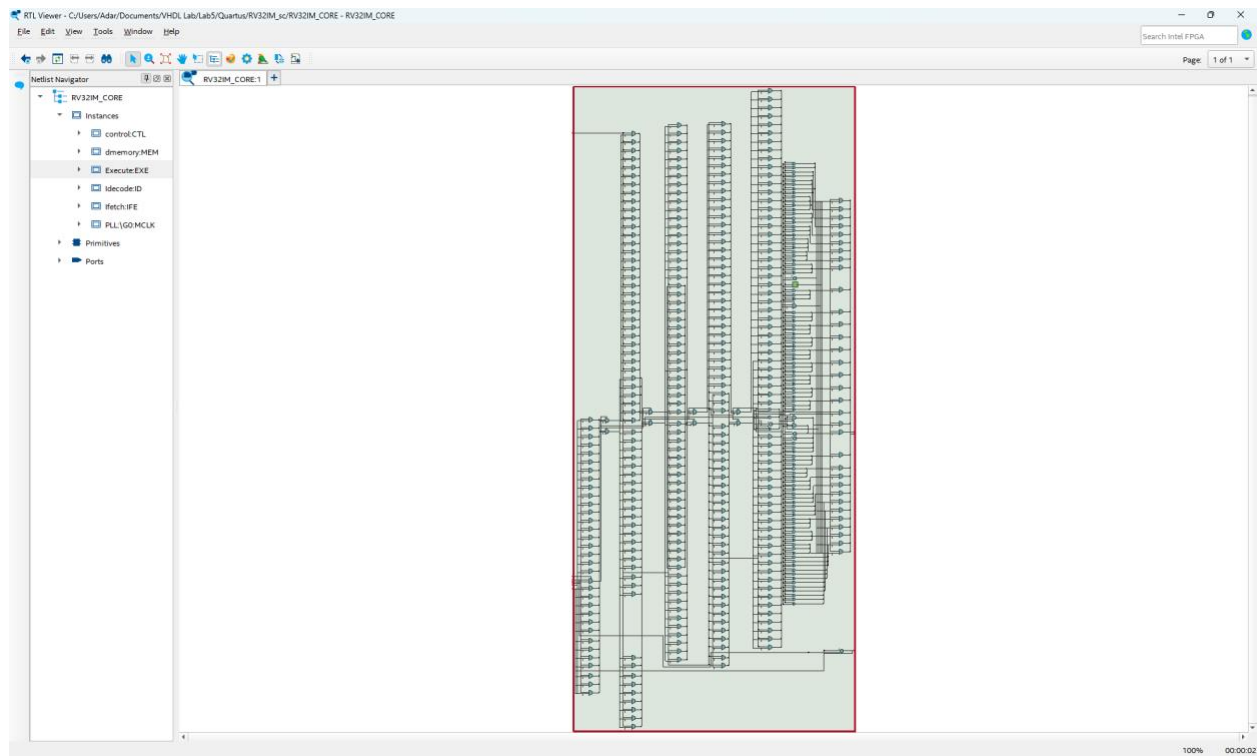
איור 7: מודול IFETCH ב-RTL Viewer: אוגר ה-PC, מחבר PC+4, מרבב ה-PC והא-ITCM



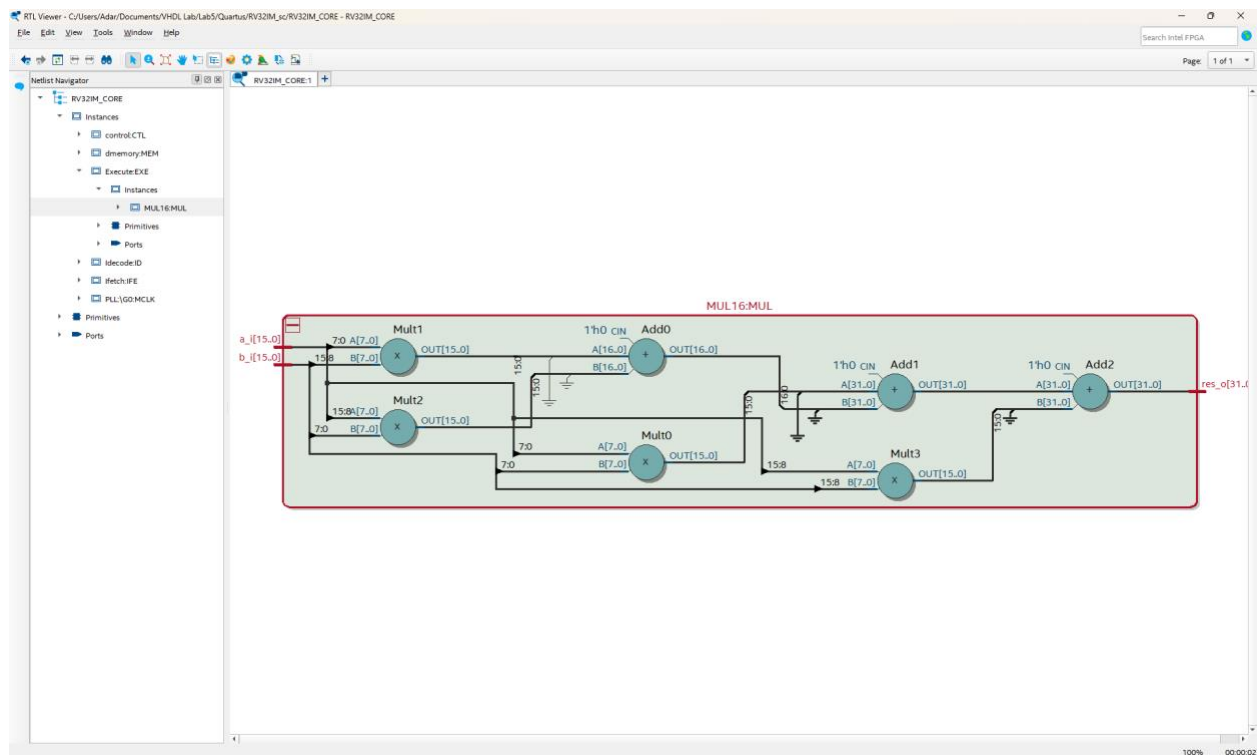
איור 8: מודול IDECODE ב-RTL Viewer: קובץ האוגרים, מחוללי ה-immediate ומרביב ה-write-back



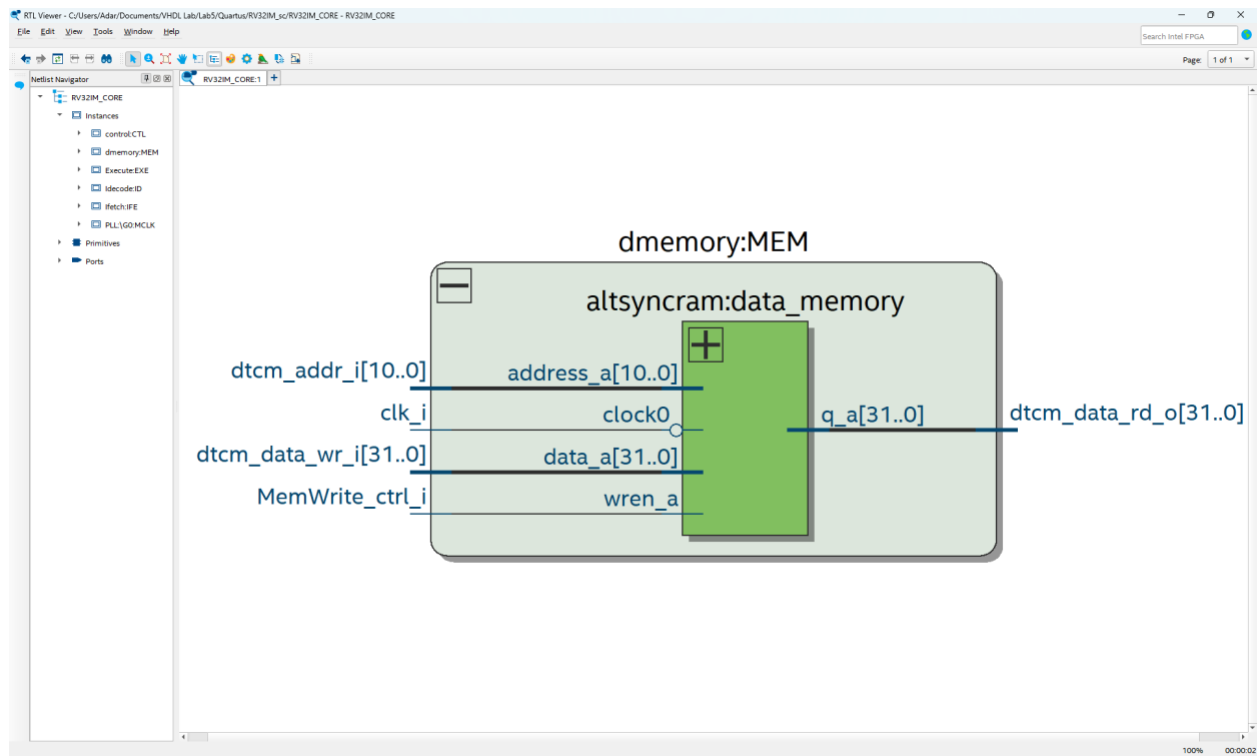
איור 9: מודול CONTROL ב-RTL Viewer: מפענח ההוראות הצירופי



איור 10: מודול EXECUTE ב-RTL Viewer, לוגיקת ההסתעפות ומופע MUL16



איור 11: מודול MUL16 ב-RTL Viewer, ארבעת המכפלים החלקיים בני 8 ביט, סכום האמצע והחיבור הסופי

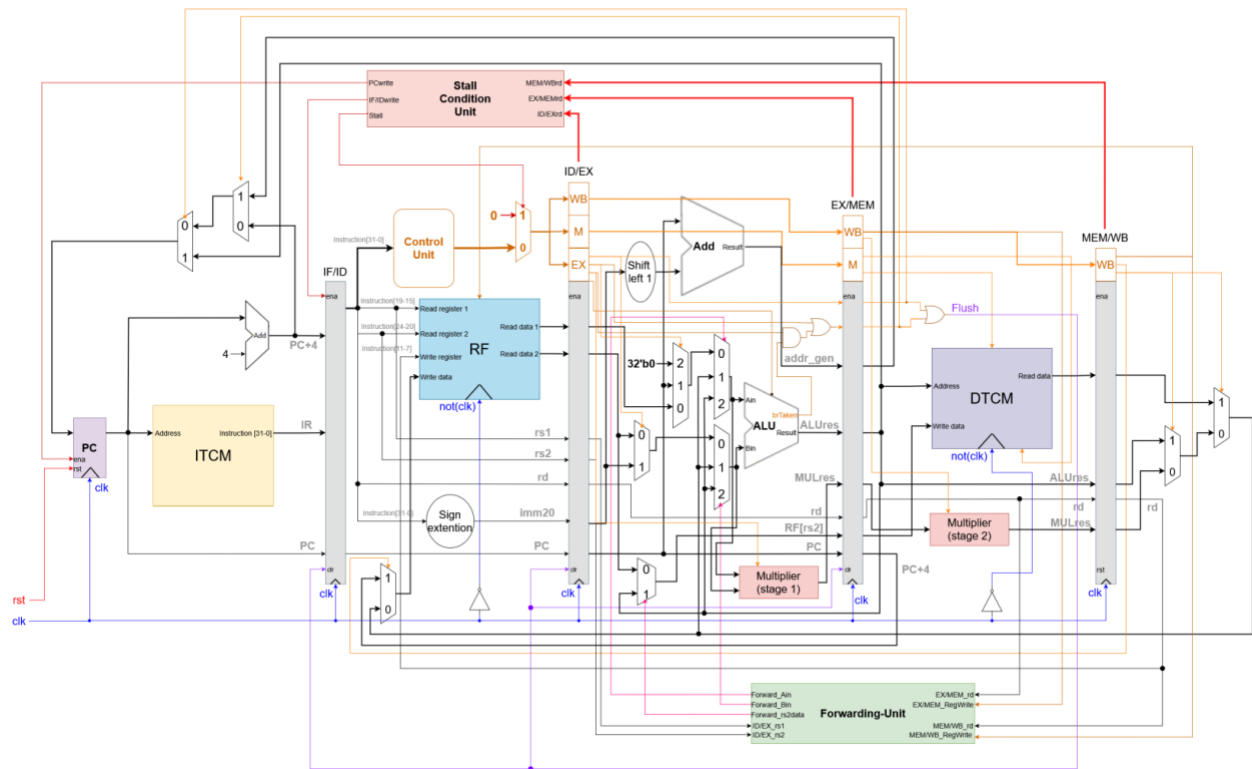


איור 12: מודול DTCM ב-RTL Viewer: ה-DTCM על בסיס altsyncram

4. חלק 2 - מיקרו-ארכיטקטורת Pipelined RV32IM

בחלק השני של המשימה הרחבנו את מיקרו-ארכיטקטורת ה-RV32IM single-cycle שתוכננה בחלק 1 למימוש pipeline סקלרי בן חמישה שלבים. במקום שהוראה אחת תעבור את כל המסלול הארוך במחזור אחד, המסלול נחתך לחמישה קטעים קצרים המופרדים באוגרי pipeline, וחמש הוראות שונות מתקדמות בו-זמנית, כל אחת בשלב אחר. תדר השעון נקבע כעת לפי השלב האיטי ביותר בלבד ולא לפי סכום כל השלבים.

RISC-V RV32I Single-Thread pipeline uArchitecture (supports the base Integer unprivileged ISA) ©Hananya Ribo



איור 13: מיקרו-ארכיטקטורת Pipelined RV32IM (MULDIV partial) בת חמישה שלבים, עם בדיקת interlock ותמיכת forwarding מלאה

4.1 חלוקה לחמישה שלבים ואוגרי ה-pipeline

את חלוקת התכן ביססנו על תת-מודולים הקיימים מחלק 1: כל תת-מודול הפך לשלב ה-pipeline, ואוגר ה-pipeline שבסופו מומש בתוך אותו מודול ולא כישות נפרדת. כך נשמר התכן המבני של הרמה העליונה, המחושבת חמישה שלבים ושתי יחידות בקרת ה-pipeline בדיוק כמו בתרשים.

#	שלב	מודול VHDL	מה מתבצע בשלב	אוגר ה-pipeline שבסופו
1	IF	IFETCH.vhd	אוגר ה-PC, מחבר PC+4, מרביב ה-redirect, קריאה מה-ITCM	IF/ID
2	ID	IDECODE.vhd CONTROL.vhd	פענוח, קריאת קובץ האוגרים, ייצור immediate והרחבת סימן	ID/EX
3	EX	EXECUTE.vhd	מרבבי forwarding, ה-ALU, MUL16, מחבר כתובת ההסתעפות	EX/MEM
4	MEM	DMEMORY.vhd	גישה ל-DTCM, הכרעת הסתעפויות וקפיצות	MEM/WB
5	WB	מרבב ה-write-back שבתוך IDECODE.vhd	בחירה בין תוצאת ה-ALU, נתון מה-DTCM ו-PC+4, וכתיבה לקובץ האוגרים	-

טבלה 4: מיפוי חמשת שלבי ה-pipeline למודולי ה-VHDL ולאוגרי ה-pipeline

קובץ האוגרים נקרא בשלב ID ונכתב בשלב WB. מכיוון ששני הצדדים נוגעים באותו מערך אוגרים, טבעי למקם את שניהם באותו מודול. פתרון זה גם מאפשר לממש בתוך המודול את עקיפת הקריאה (read bypass): כאשר הוראה בשלב WB כותבת לאותו אוגר שהוראה בשלב ID קוראת ממנו, הקריאה מקבלת את הערך החדש. בלי עקיפה זו היה נדרש עוד מנגנון forwarding או עוד stall.

4.2 data hazards ויחידת ה-forwarding

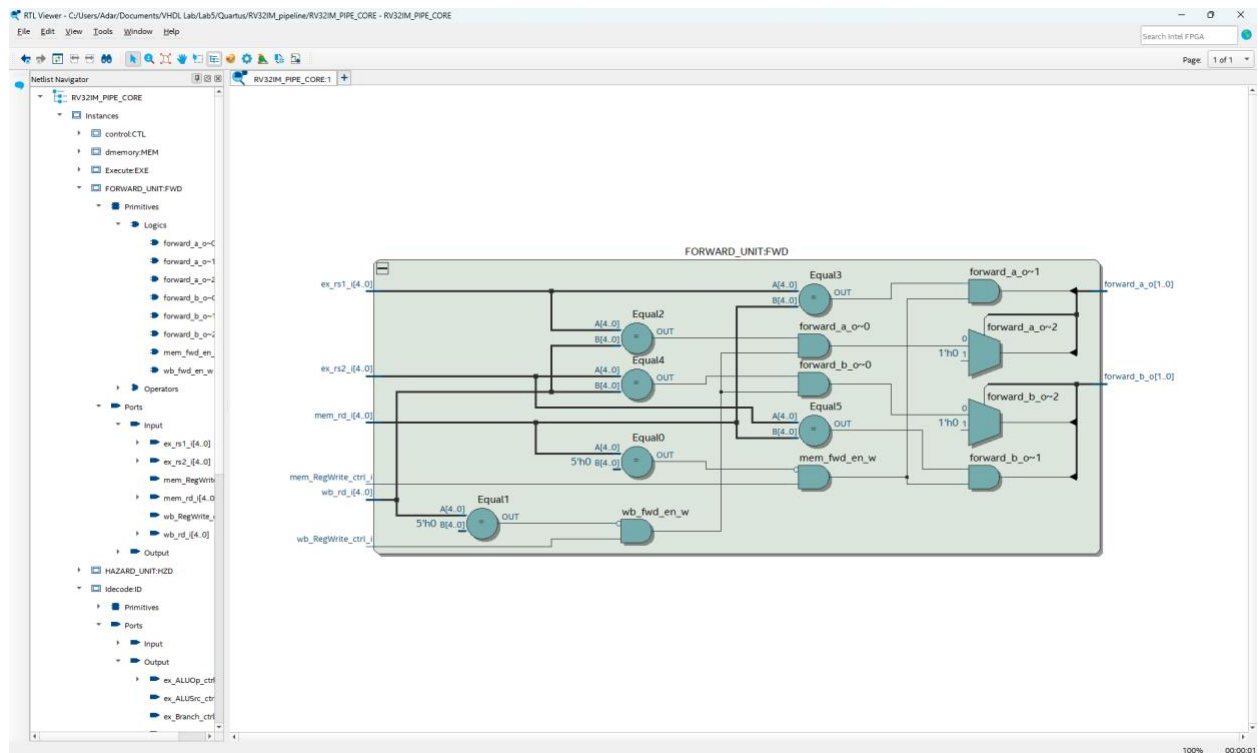
data hazard מסוג RAW מתרחש כאשר הוראה זקוקה לתוצאה של הוראה קודמת שטרם נכתבה לקובץ האוגרים. במקום להשהות את ה-pipeline, יחידת ה-forwarding מזרימה את התוצאה ישירות מאוגר ה-pipeline שבו היא כבר קיימת אל כניסות ה-ALU.

היחידה FORWARD_UNIT.vhd משווה את rs1 ו-rs2 של ההוראה שבשלב EX מול אוגרי היעד של ההוראות שבשלבים MEM ו-WB, ומניעה שני אותות בחירה בני שני ביט:

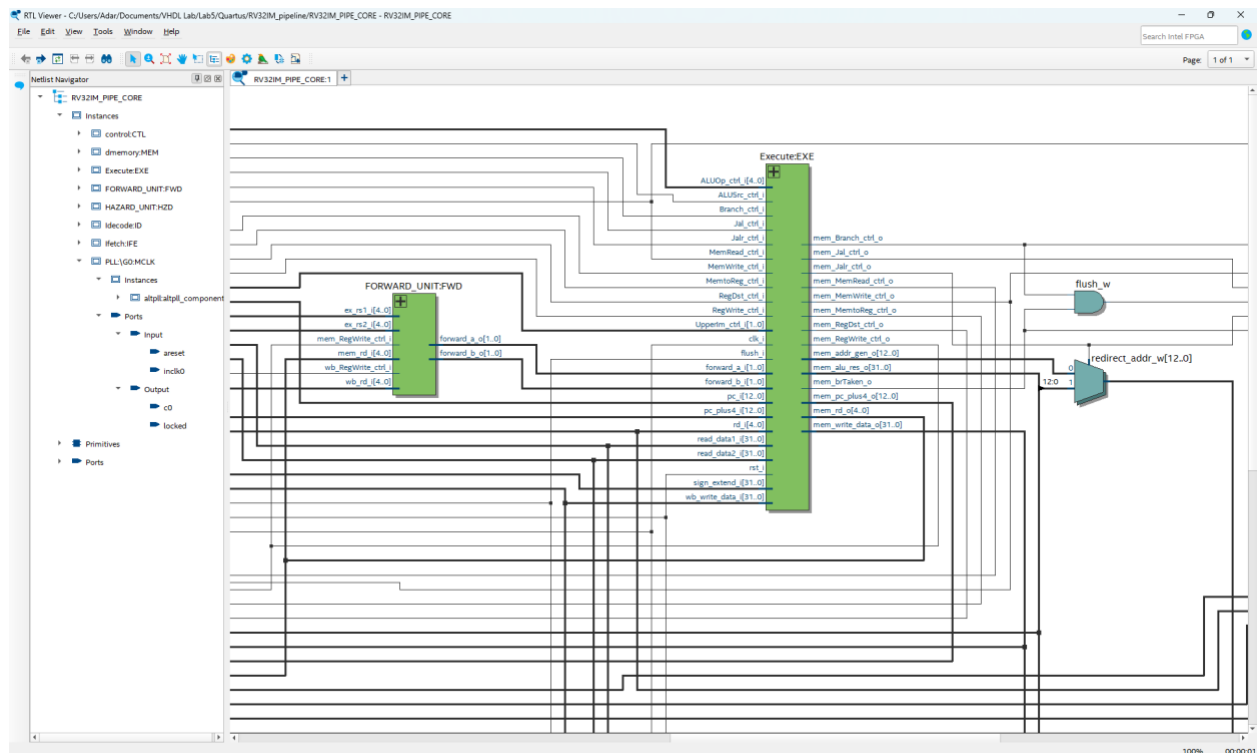
קידוד	שם	מקור הנתון	מתי
00	FWD_NONE	אוגר ה-ID/EX (הערך שנקרא מקובץ האוגרים)	אין תלות
10	FWD_MEM	תוצאת ה-ALU שבאוגר EX/MEM	תלות במרחק 1
01	FWD_WB	נתון ה-write-back שבאוגר MEM/WB	תלות במרחק 2

טבלה 5: קידוד מרבבי ה-forwarding ביחידת FORWARD_UNIT

שני תנאים נוספים חייבים להתקיים כדי שהנתון יוזרם: ההוראה הקודמת אכן כותבת לאוגר (RegWrite = '1', והיעד אינו x0). האוגר x0 קשיח לאפס בארכיטקטורת RISC-V, ולכן forwarding אליו היה שגוי. כמו כן נקבעה עדיפות ל-EX/MEM על פני MEM/WB: אם שתי ההוראות שלפנינו כותבות לאותו אוגר, הערך הנכון הוא של המאוחרת מביניהן, לפי הכלל "הכותב האחרון מנצח".



איור 14: RTL Viewer - יחידת FORWARD_UNIT: משווי אוגרי היעד ולוגיקת העדיפות המניעה את forward_a ו-forward_b_o



איור 15: RTL Viewer - חיבור FORWARD_UNIT אל EXECUTE: אותות forward_a/forward_b נכנסים למרבבי כניסות ה-ALU. מימין נראים שער ה-flush_w ומרבב ה-redirect_addr_w של סעיף 4.4

4.3 load-use hazard ויחידת interlock

forwarding מלא מכסה כמעט את כל ה-data hazards, אך נשאר אחד שאין דרך לכסות: הוראת lw מקבלת את הנתון שלה מה-DTCM רק בסוף שלב MEM, בעוד שההוראה הצמודה אחריה זקוקה לו כבר בתחילת שלב EX שלה, כלומר באותו מחזור עצמו. אין דרך פיזית להזרים נתון אחורה בזמן, ולכן נדרש stall של מחזור אחד.

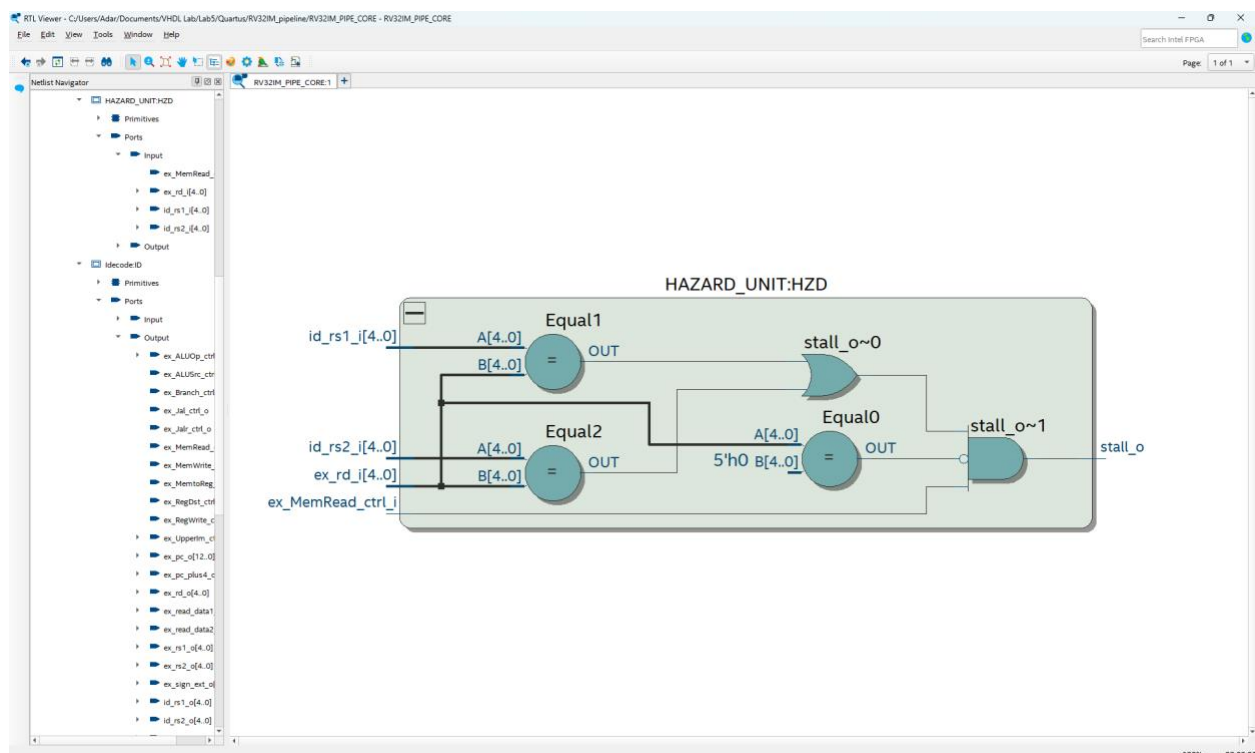
היחידה HAZARD_UNIT.vhd מממשת בדיקה צירופית טהורה (combinational check, בניגוד לגישת score-board), ללא מצב פנימי כלשהו:

$$\text{stall} = \text{ex_MemRead} \text{ AND } (\text{ex_rd} \neq x0) \text{ AND } (\text{ex_rd} = \text{id_rs1} \text{ OR } \text{ex_rd} = \text{id_rs2})$$

כאשר $\text{stall} = '1'$, הרמה העליונה מפעילה interlock של מחזור אחד:

- המודול IFETCH מקפיא את ה-PC ואת אוגר IF/ID, כך שההוראה הצורכת מפוענחת שוב בשלב ID במחזור הבא.
- המודול DECODE מזריק bubble לאוגר ID/EX.
- הוראת ה-1w עצמה ממשיכה כרגיל במורד ה-pipeline.

מחזור stall אחד מספיק. אחריו ההוראה הצורכת עדיין בשלב ID בעוד ה-1w נמצאת ב-MEM. במחזור שאחרי הצורכת מגיעה ל-EX וה-1w נמצאת ב-WB, ואז נתון הטעינה מוזרם אליה על ידי FORWARD_UNIT בבחירה FWD_WB. שאר המרחקים מכוסים ממילא: צרכן במרחק 2 פוגש את ה-1w ב-WB בלי stall, וצרכן במרחק 3 קורא את קובץ האוגרים ב-ID בדיוק כשה-1w כותבת אליו ב-WB, מקרה שמכוסה על ידי עקיפת הקריאה שבתוך DECODE.



איור 16: RTL Viewer - יחידת HAZARD_UNIT: הבדיקה הצירופית המזהה load-use ומניעה את stall_o

4.4 control hazards: הכרעת הסתעפויות בשלב MEM

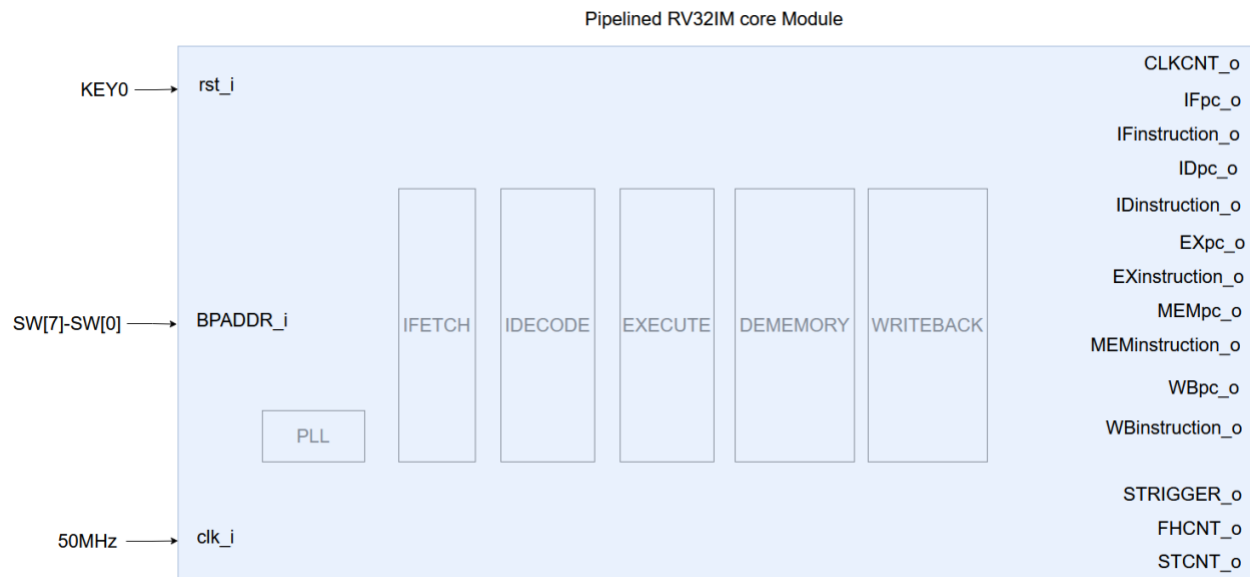
בתכן זה, הסתעפויות מותנות וקפיצות בלתי-מותנות (jalr, jal) מוכרעות בשלב הרביעי של ה-pipeline, MEM. בזמן שההוראה המסתעפת מתקדמת בין IF ל-MEM, ה-pipeline ממשיכה לשלוף באופן ספקולטיבי את ההוראות העוקבות בזיכרון. כאשר מתברר שההסתעפות נלקחת, שלוש ההוראות הצעירות ממנה, אלו שבאוגרי ID/EX, IF/ID ו-EX/MEM, שגויות ויש לבטלן. מכאן שעומק ה-flush הוא 3, וזהו המקדם depth המופיע בנוסחת ה-IPC שבסעיף 4.5.

כתובת ה-redirect (redirect_addr_w) נבחר מתוך אוגר ה-EX/MEM: עבור branch ו-jal זוהי תוצאת מחבר כתובת ההסתעפות, ועבור jalr זוהי תוצאת ה-ALU. מרבב זה ושער ה-flush_w מופיעים באיור 15 שלמעלה.

ייתכן מצב שבו stall ו-flush פעילים באותו מחזור. במקרה כזה ה-flush גובר: ההוראה שגרמה ל-redirect נמצאת ב-MEM ולכן היא מבוגרת מההוראה שגרמה ל-stall, וממילא ההוראה הצעירה עומדת להתבטל, וה-stall חסר משמעות. כלל קדימות זה מונע איבוד של אירוע redirect.

4.5 תוספות ניפוי שגיאות ו-Signal-Tap

לצורך ולידציה על הכרטיס וחישוב IPC, הישות העליונה של ה-pipeline מרחיבה את זו של ה-single-cycle בארבע תוספות המוגדרות בסעיף 6 של המשימה.



איור 17: הישות העליונה של ה-pipeline - RV32IM_PIPE_CORE עם מוני CLKCNT, STCNT, FHCNT ואוגר נקודת העצירה BPADDR

תפקיד	רוחב	אות
מונה מחזורי שעון מאז האיפוס - המכנה בנוסחת ה-IPC.	16	CLKCNT_o
מונה stalls; מתקדם בעליית השעון בכל מחזור שבו stall = '1'. מתאפס באיפוס.	8	STCNT_o
מונה flush; מתקדם בכל אירוע redirect. מתאפס באיפוס.	8	FHCNT_o
אוגר כתובת נקודת עצירה (ברזולוציית מילה), מוזן מ-SW7-SW0, שעל הכרטיס.	8	BPADDR_i
אות ההדק ל-Signal-Tap, המופק מהשוואת ה-PC של שלב IF ל-BPADDR_i.	1	BPTRIGGER_o

טבלה 6: אותות הניפוי שנוספו בישות העליונה של ה-pipeline

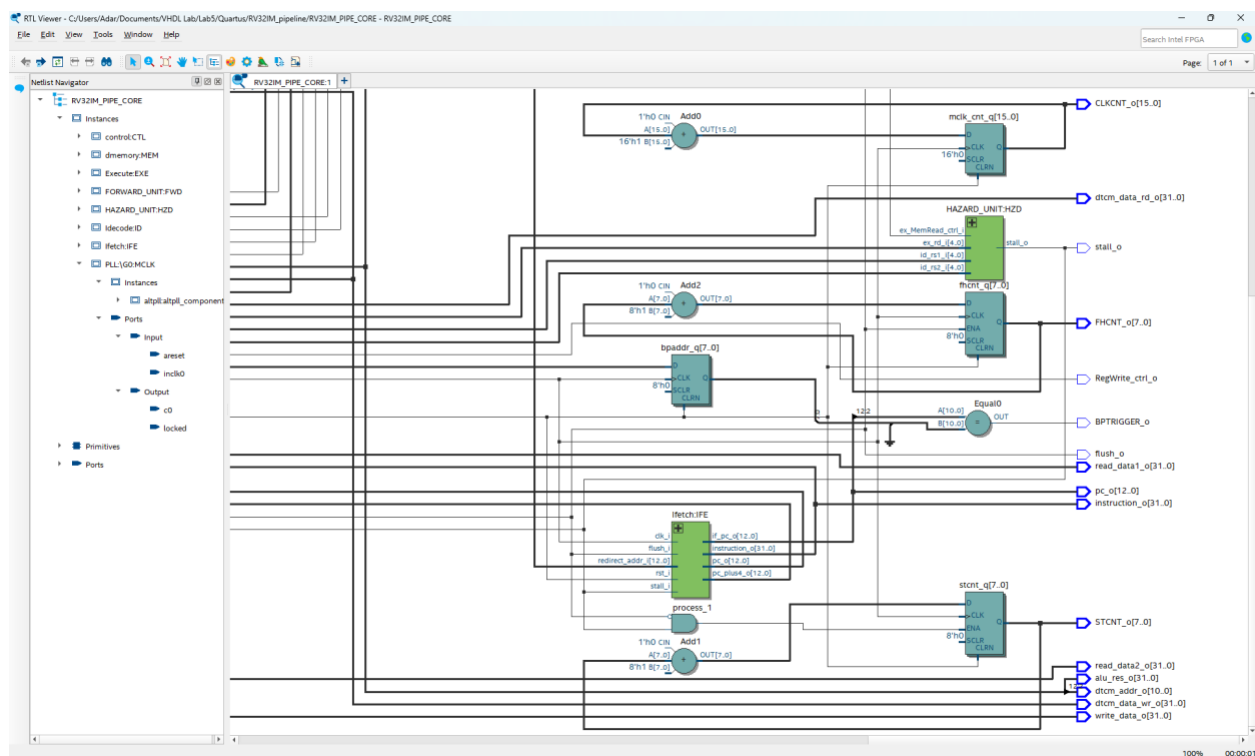
שתי המשוואות המגדירות את המנגנון:

$$\text{SignalTap}_{\text{trigger}} = (\text{IFpc} == \text{BPADDR}_i)$$

$$\text{IPC} = \left[\text{CLKCNT}_o - (\text{STCNT}_o + 4 + \text{depth} \cdot \text{FHCNT}_o) \right] / \text{CLKCNT}_o = \text{InstructionCounter} / \text{CLKCNT}_o, \text{IPC} = 1 / \text{CPI}$$

איברי המונה מתפרשים כך: מכלל מחזורי השעון מחסירים את מחזורי ה-stall (STCNT_o), את 4 המחזורים של מילוי ה-pipeline בתחילת הריצה (pipeline fill): ההוראה הראשונה זקוקה ל-5 מחזורים כדי לפרוש, ולכן 4

מחזורים אינם מנוצלים), ואת המחזורים שאבדו בכל אירוע redirect, 3 לכל flush, כמוסבר בסעיף 4.4. מה שנותר הוא מספר ההוראות שפרשו בפועל.



איור 18: RTL Viewer - לוגיקת הניפוי ברמה העליונה: המונים `stcnt_q`, `clkcnt_q`, `fhcnt_q` עם המחברים והכניסות `ENA`, אוגר `bpaddr_q` והמשווה `Equal0` המפיק את `BPTRIGGER_o`

יכולת ה-Signal-Tap מוגבלת על ידי הזיכרון המשובץ הפנוי ברכיב, לפי תנאי הזיכרון הכולל:

$$ST_{depth} = (\text{Embedded memory size} - \text{Design memory usage}) / \#ST_{channels}$$

התכן שלנו צורך 131,072 ביט מתוך 3,981,312 ביט זמינים (סעיף 6.1), ולכן נותרים 3,850,240 ביט לרשות ה-Signal-Tap. זה מספיק כדי לדגום מספר גדול של ערוצים בעומק גדול.

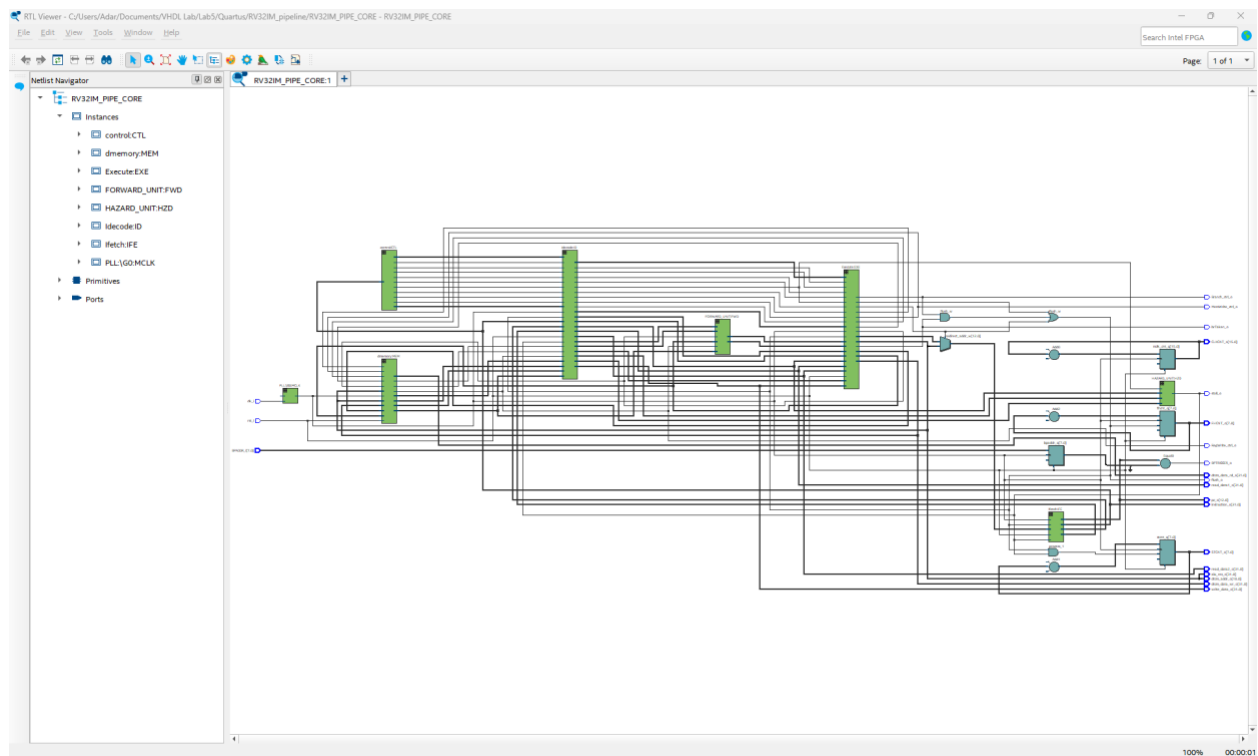
4.6 תיאור קבצי ה-HDL

קובץ	תיאור
cond_compilation_package.vhd const_package.vhd PLL.vhd, MUL16.vhd	זהים בתפקידם לקבצים המקבילים בחלק 1 (סעיף 3.4). MUL16 ממשיך לשמש בשלב EX ללא שינוי.
IFETCH.vhd	שלב IF ואוגר IF/ID. ה-PC קיבל כניסת enable (הקפאה בזמן stall), flush_i מזריק bubble, ומרכב ה-PC הבא מקבל את כתובת ה-redirect משלב MEM.
IDCODE.vhd	שלב ID ואוגר ID/EX (נתונים, immediates, rd/rs2/rs1, ואותות בקרה). קובץ האוגרים נכתב משלב WB בעוד הקריאות מתבצעות ב-ID.
CONTROL.vhd	אותו מפענח צירופי מחלק 1, היושב בשלב ID; פלטיו נעים במורד ה-pipeline יחד עם ההוראה.
EXECUTE.vhd	שלב EX ואוגר EX/MEM; מרבכי forwarding על שתי כניסות ה-ALU (ID/EX מול EX/MEM מול נתון ה-WB), המונעים על ידי FORWARD_UNIT.

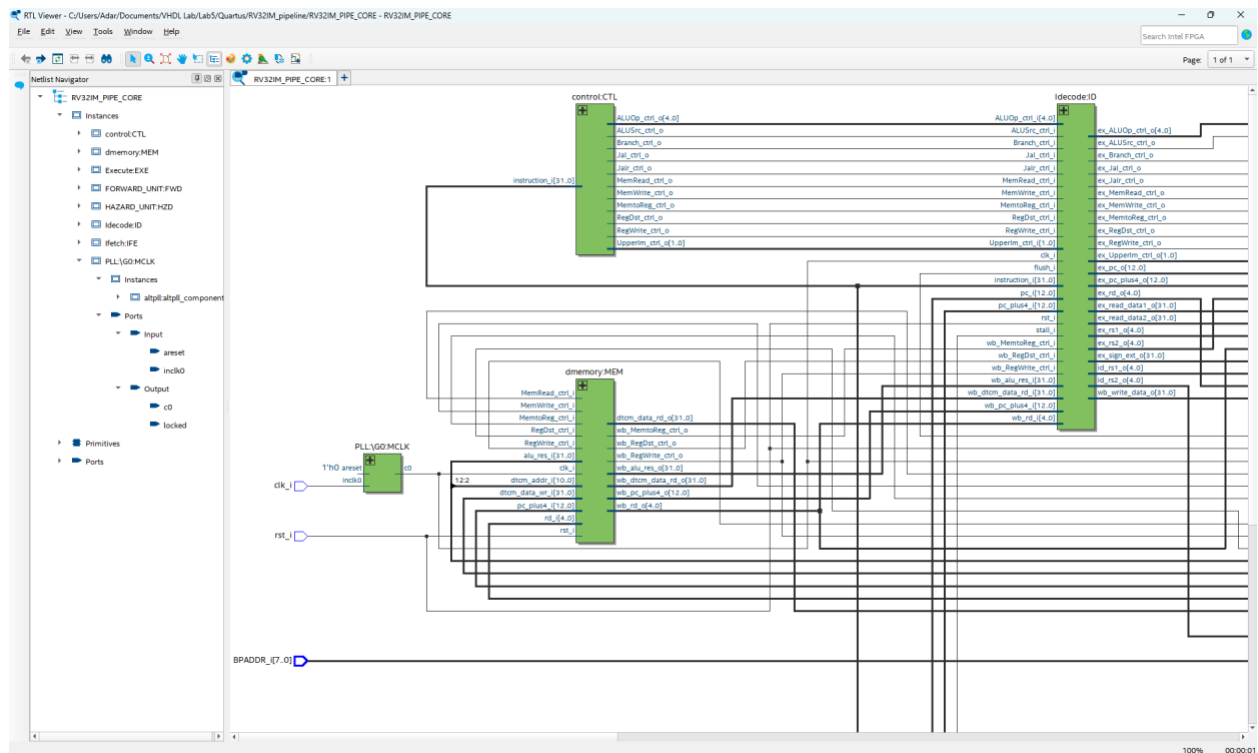
קובץ	תיאור
DMEMORY.vhd	שלב MEM (ה-DTCM) ואוגר MEM/WB.
HAZARD_UNIT.vhd	interlock צירופי ל-load-use: משווה את rs2/rs1 של שלב ID מול אוגר היעד שבשלב EX, ומשהה מחזור אחד.
FORWARD_UNIT.vhd	משווה את rs2/rs1 של שלב EX מול אוגרי היעד שב-MEM/EX וב-MEM/WB (בתנאי '1' RegWrite = '1' ו-rd ≠ x0) ומניע את אותות בחירת מרבבי ה-forwarding.
RV32IM_PIPE_CORE.vhd	הישות העליונה המבנית; הסתעפויות וקפיצות מוכרעות ב-MEM (flush בעומק 3). כוללת את מוני CLKCNT, STCNT ו-FHCNT ואת אוגר BPADDR המשווה ל-PC של שלב IF לצורך BPTRIGGER.

טבלה 7: תיאור קבצי ה-HDL של מימוש ה-pipeline (תיקיה RV32IM_pipeline/DUT)

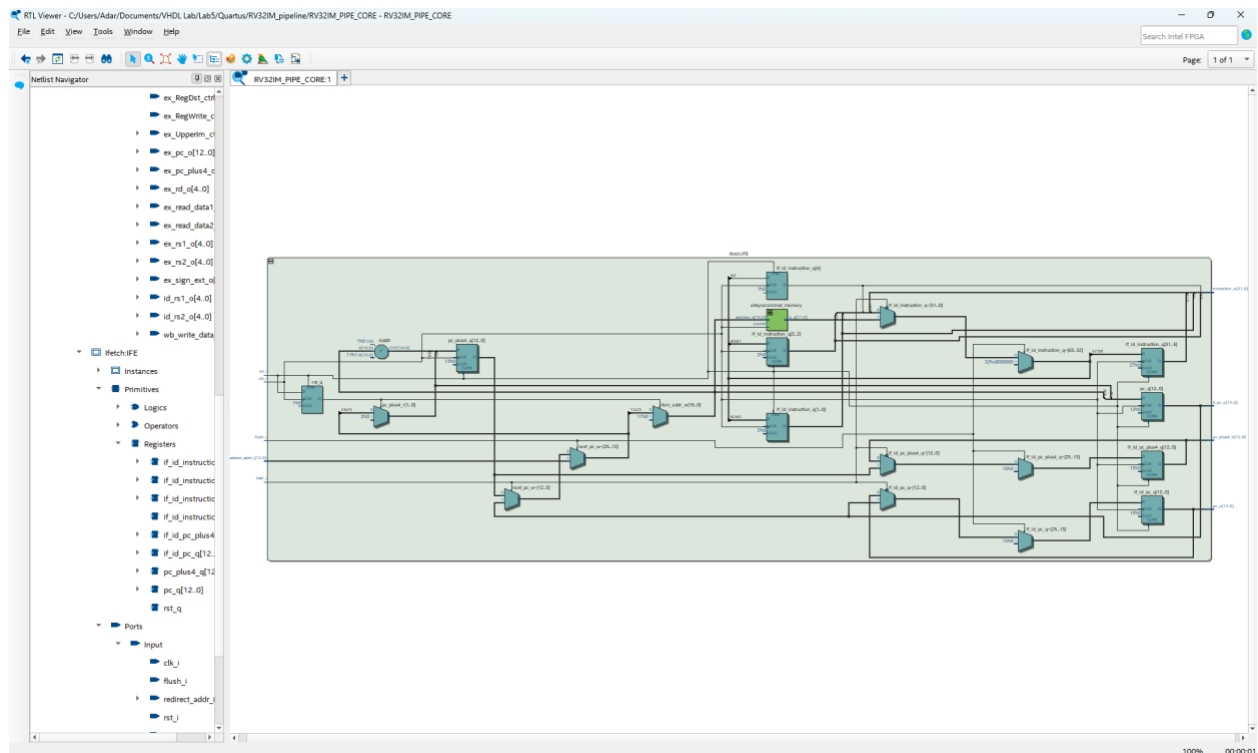
4.7 תוצאות RTL Viewer



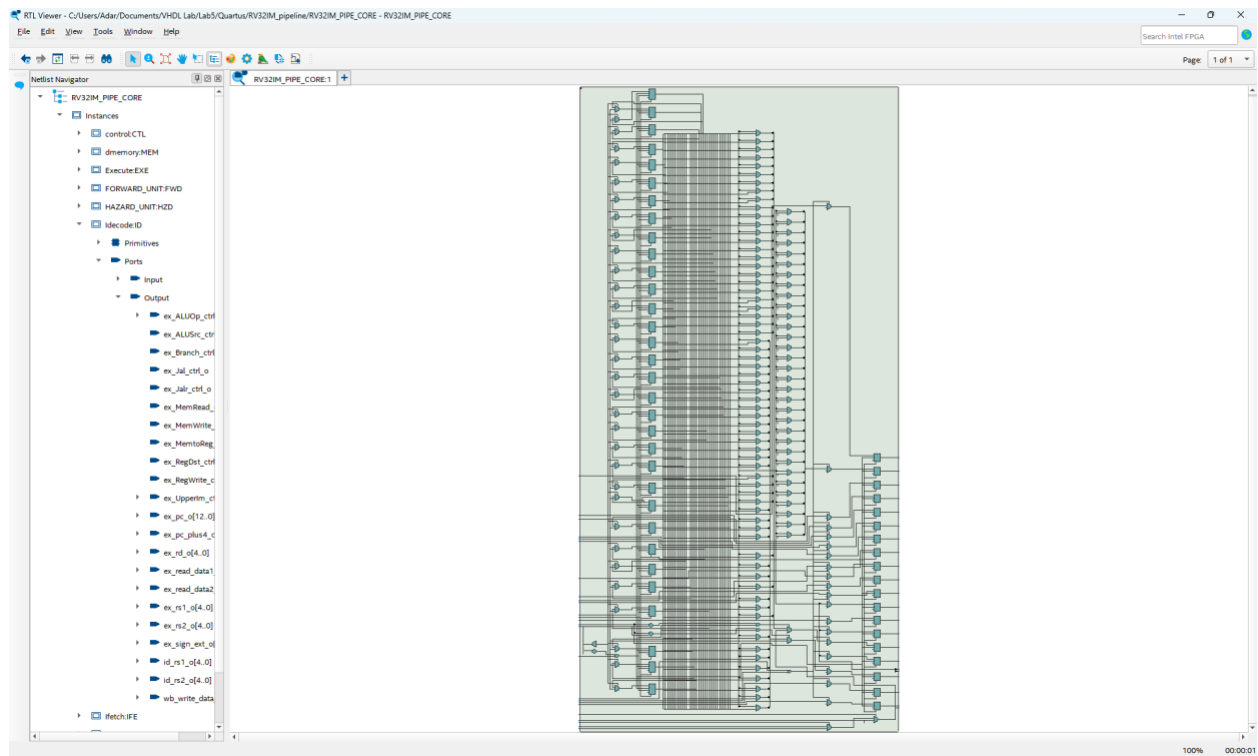
איור 19: RTL Viewer - הרמה העליונה של RV32IM_PIPE_CORE. ב-Netlist Navigator נראים שמות המופעים: HAZARD_UNIT:HZD, FORWARD_UNIT:FWD, Execute:EXE, dmemory:MEM, control:CTL, PLL:\G0:MCLK ו-Ifetch:IFE, Idecode:ID



איור 20: RTL Viewer - הגדלה של הרמה העליונה: CONTROL, IDECODE, DMEMORY וה-PLL. נראה בה מסלול ה-
 write-back, כלומר אותות wb_* החוזרים מאוגר ה-MEM/אל IDECODE, וכן פלטי אוגר ה-ID/EX (ex_*)



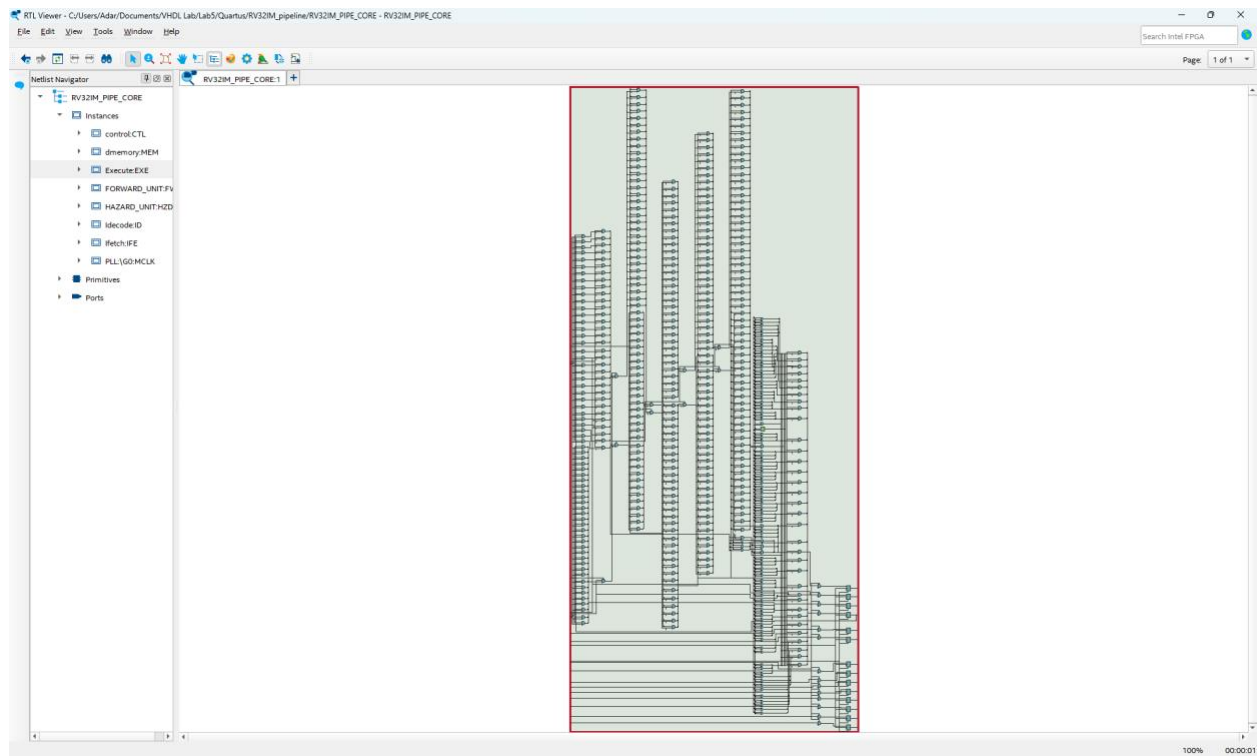
איור 21: RTL Viewer - שלב IF: אוגר ה-PC עם כניסת enable, מרבב ה-redirect, ה-ITCM ואוגר IF/ID



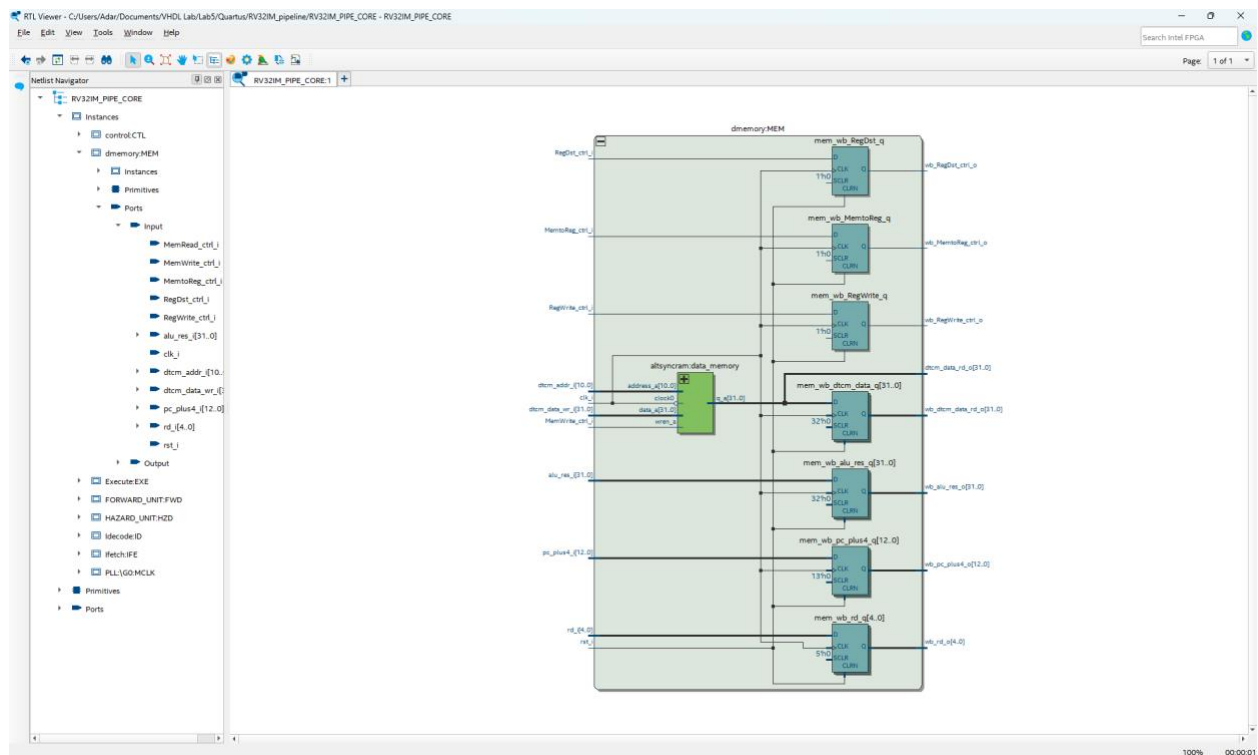
איור 22: RTL Viewer - שלב ID: קובץ האוגרים, מחוללי ה-immediate, מרביב ה-write-back ואוגר ID/EX



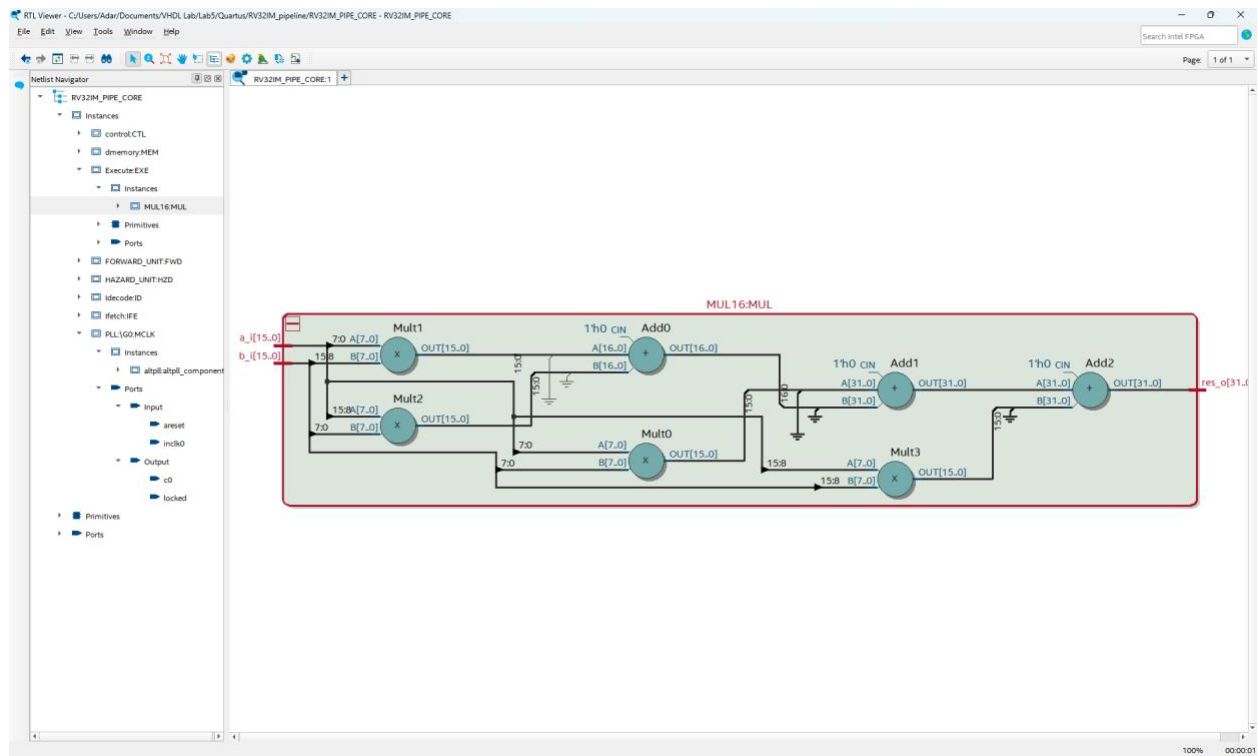
איור 23: RTL Viewer - מודול CONTROL בשלב ID



איור 24: RTL Viewer - שלב EX: מרבבי ה-forwarding, ה-ALU, מופע MUL16 ואוגר EX/MEM



איור 25: RTL Viewer - שלב MEM: ה-DTCM, לוגיקת הכרעת ההסתעפות ואוגר MEM/WB



איור 26: RTL Viewer - מודול MUL16 בתוך שלב ה-EX של ה-pipeline

5. וריפיקציה וולידציה של התכן

5.1 מתודולוגיית האימות

האימות בוצע לפי זרימת התכן שבסעיף 8 של המשימה, עם RARS כמודל זהב. עבור כל אפליקציית benchmark, קובץ ה-DTCM.h שמייצר RARS בסוף הריצה משמש כתוצאה הנכונה, ומולו משווה קובץ ה-DTCM.mem שנשפך מהסימולציה ב-ModelSim.

קובץ *.do	תפקיד
compile.do	יוצר את ספריית work ומקמפל את החבילות, ה-DUT וה-testbench בסדר תלויות נכון, בתקן VHDL-2008.
run_test.do	טוען את תמונות ה-benchmark הנבחר, מפעיל את הסימולציה עם wave.do, עוצר בסיום התוכנית ושופך את ה-DTCM.
wave.do	הגדרת האותות בחלון הגלים, מקובצים לפי PIPE_CTRL, TOP, MUL, CONTROL, COUNTERS ו-REGFILE.
mem_dump.do	מייצא את ה-DTCM לקובץ DTCM.mem במבנה של מילה הקסדצימלית אחת בשורה - בדיוק כמבנה ה-DTCM.h של RARS, כדי לאפשר השוואה ישירה.
batch_verify.do	(ב-pipeline) הרצה אוטומטית של test1-test4; מדווחת CLKCNT/STCNT/FHCNT ו-IPC לכל טסט וכותבת DTCM_testN.mem.

טבלה 8: סקריפטי ה-ModelSim ששימשו לאימות

אפליקציות ה-benchmark מסתיימות בלולאת (1) while אינסופית. במימוש ה-single-cycle מספיק לזהות את ההוראה הזו בשלב הפענוח, אבל ב-pipeline זיהוי כזה שגוי. ההסתעפויות מוכרעות ב-MEM, ולכן השליפה הספקולטיבית מביאה את הוראת הסיום לשלב ID כבר באיטרציה הראשונה של לולאה כלשהי. ההוראה אמנם תבוטל ב-flush, אך התנאי כבר היה עוצר את הריצה מוקדם מדי. לכן השתמשנו בקריטריון אחר: אירוע flush שיעדד הפניה שלו שווה ל-PC של ההוראה המפנה עצמה ($target + 4 == mem_pc_plus4$). תנאי כזה יכול להתקיים רק עבור קפיצה עצמית.

5.2 תוצאת ההשוואה מול מודל הזהב

כל ארבע האפליקציות הורצו בשני המימושים. תמונות זיכרון הנתונים שנשפכו נבדקו הן מול DTCM.h של RARS והן זו מול זו:

benchmark	גודל התמונה	Single-Cycle מול RARS	pipeline מול RARS	Single-Cycle מול pipeline
test1	1024 מילים	זהה	זהה	זהה בית־בבית
test2	1024 מילים	זהה	זהה	זהה בית־בבית
test3	1024 מילים	זהה	זהה	זהה בית־בבית
test4	1024 מילים	זהה	זהה	זהה בית־בבית

טבלה 9: תוצאות השוואת ה-DTCM של ארבע האפליקציות בשני המימושים מול מודל הזהב

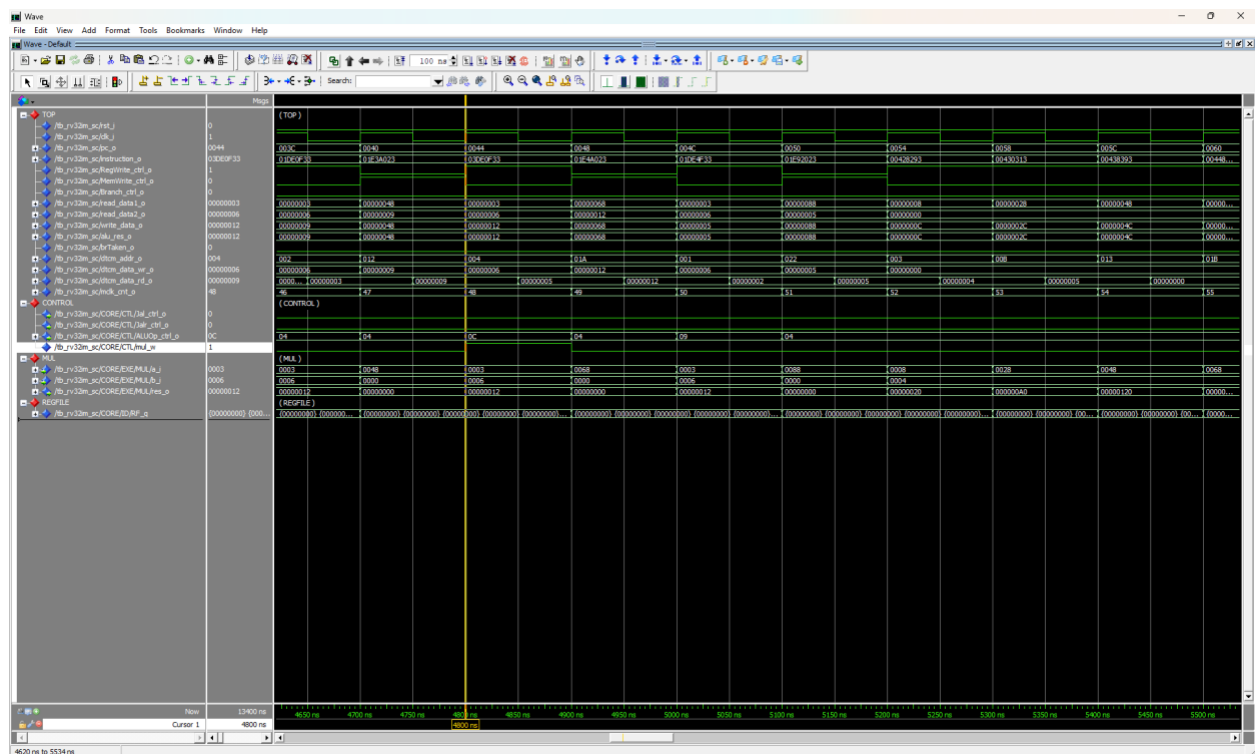
ההשוואה הישירה בין DTCM_testN.mem של ה-single-cycle לבין זה של ה-pipeline העלתה זהות מוחלטת בכל 1024 המילים ובכל ארבעת הטסטים. מכאן ששני המימושים שקולים פונקציונלית: מנגנוני ה-pipeline, כלומר

forwarding, interlock ל-load-use, ביטול הוראות ספקולטיביות והכרעת הסתעפויות ב-MEM, אינם משנים ולו ביט אחד בתוצאה הסופית של התוכנית.

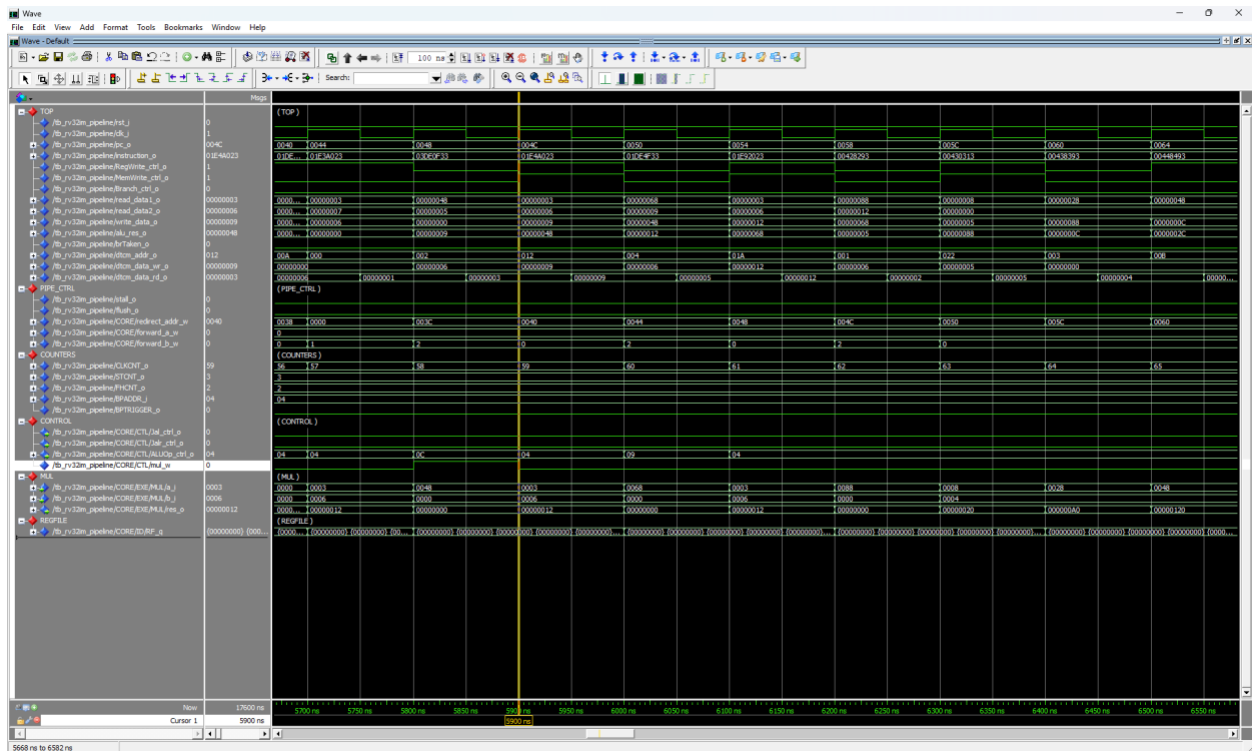
5.3 ניתוח דיאגרמות גלים

מחזור השעון ב-testbench הוא 100 ns, ולכן זמן הסימולציה מתורגם ישירות למחזורים. על בסיס זה ניתחנו את דיאגרמות הגלים של ארבע האפליקציות בשני המימושים ואת התנהגות התזמון של המערכת.

5.3.1 test1: פעולות בסיסיות והפעלת הכופל



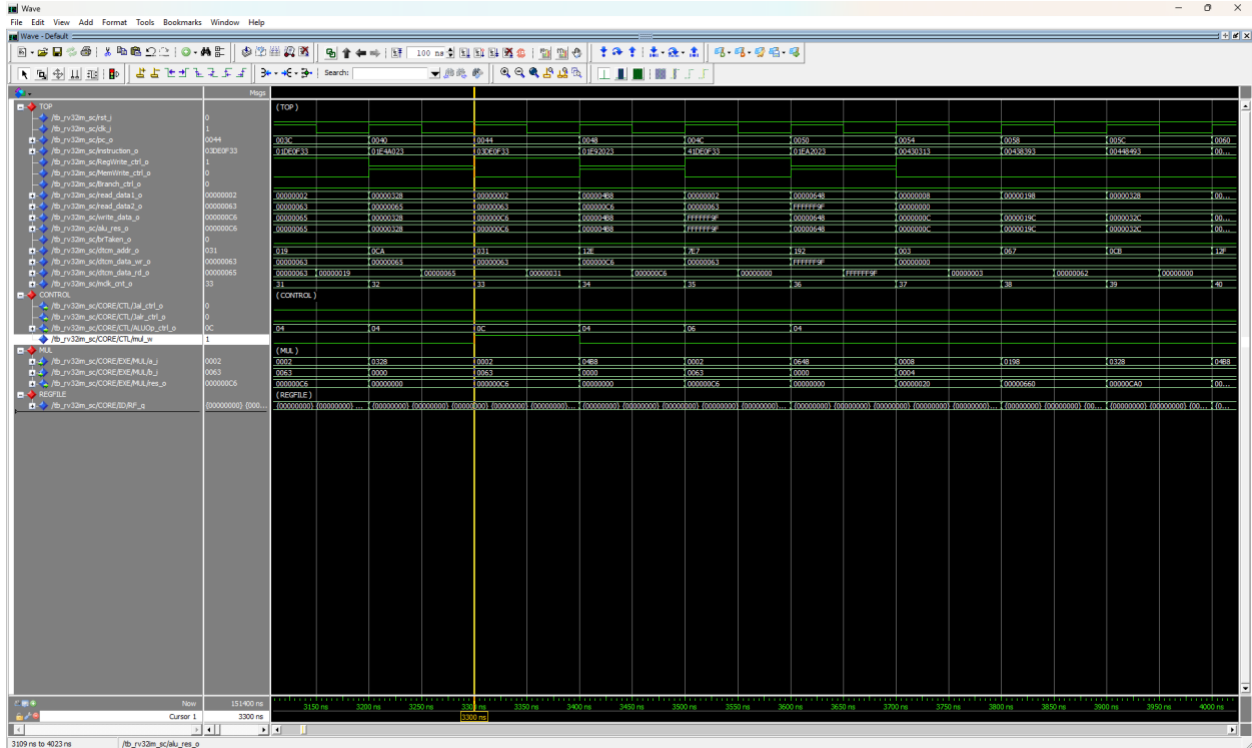
איור 27: test1 במימוש ה-single-cycle, מחזורים 46–55. בכל מחזור pc_o מתקדם ב-4 והוראה חדשה מסתיימת, ו-mul_w = '1' ו-mul_u = 0C, ופלט MUL/res_o שמתקבל באותו מחזור



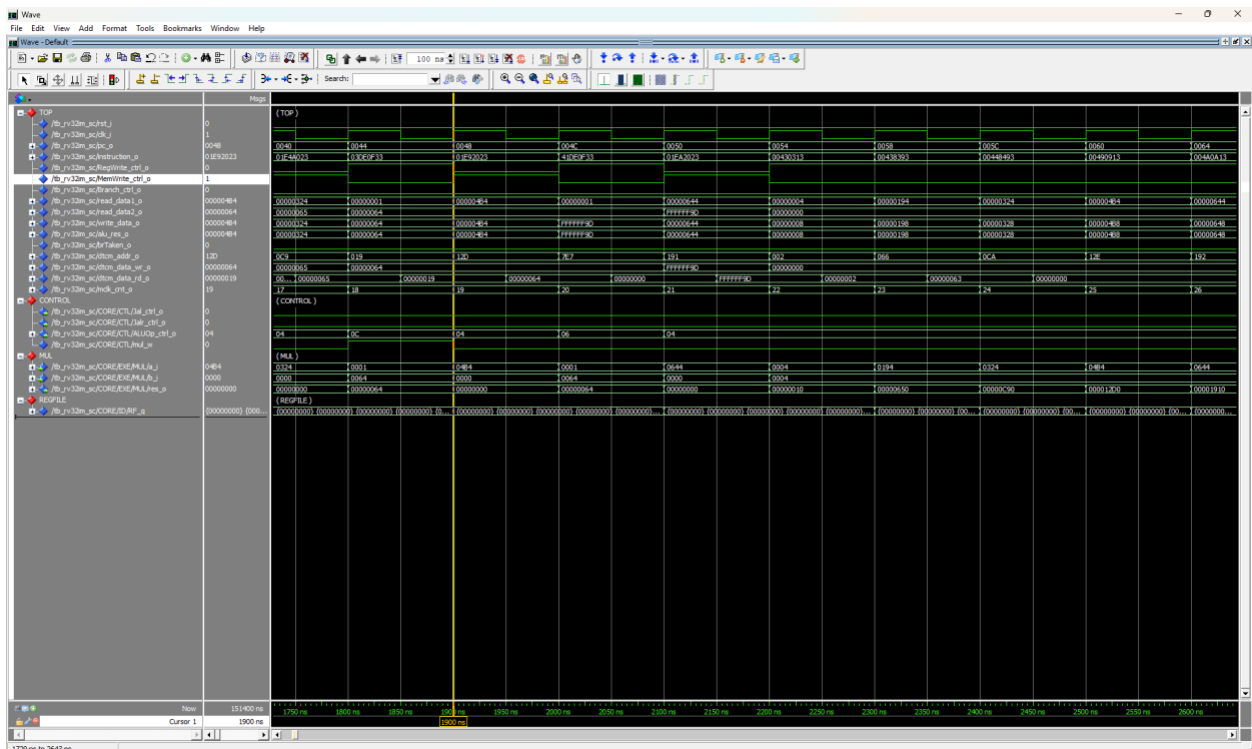
איור 28: test1 במימוש ה-pipeline, סביב מחזור 59 ($FHCNT = 2, STCNT = 3, CLKCNT = 59$). האות $forward_b_w$ מתחלף בין 0, 1 ו-2, כלומר ה-forwarding אכן פועל: הוראות עוקבות מקבלות את אופרנדיהן מאוגרי MEM/WB ו-EX/MEM במקום מקובץ האוגרים

בין שתי הדיאגרמות יש הבדל בקריאה עצמה: במימוש ה-single-cycle כל שורות האותות מתייחסות לאותה הוראה בכל רגע נתון, ואילו ב-pipeline כל שורה מתייחסת לשלב אחר. האות pc_o שייך לשלב IF, $instruction_o$ לשלב ID, $read_data2_o/read_data1_o$ לשלב EX, alu_res_o ואפיקי ה-DTCM לשלב MEM, ו- $write_data_o$ לשלב WB, כך שבכל צילום מסך נראות חמש הוראות שונות בו-זמנית.

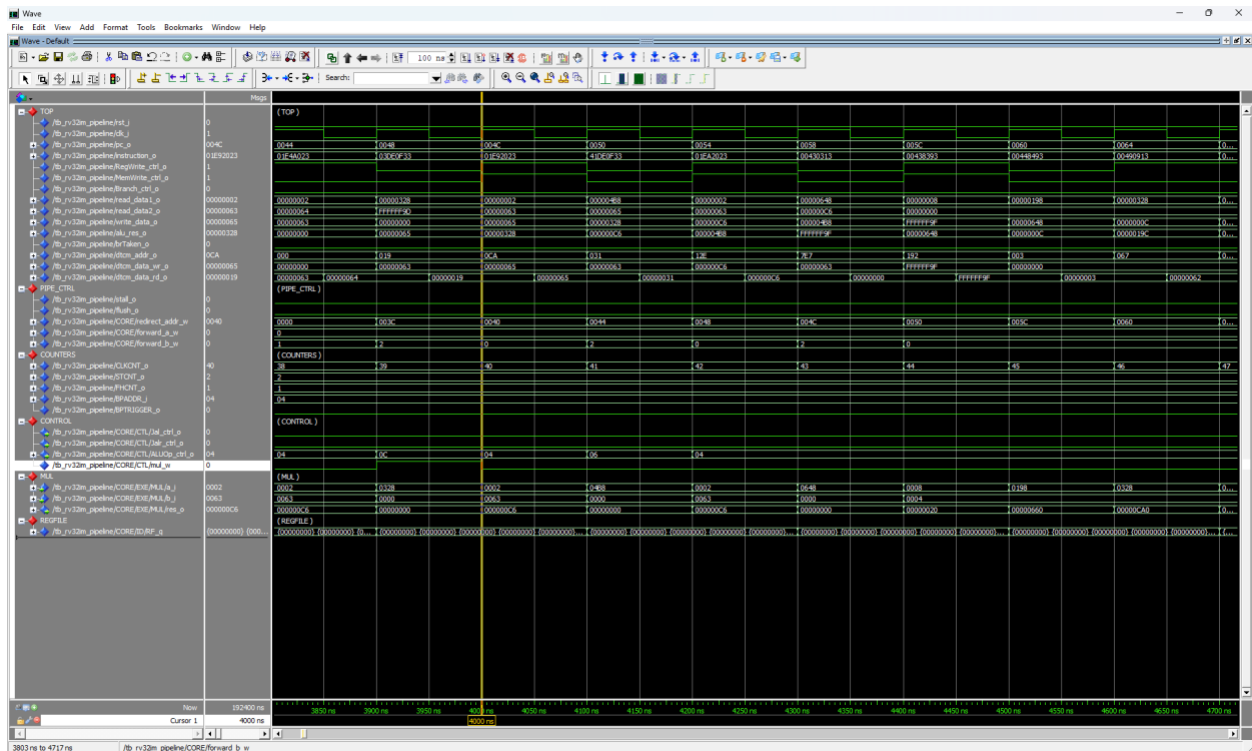
load-use stalls-1 forwarding :test2 5.3.2



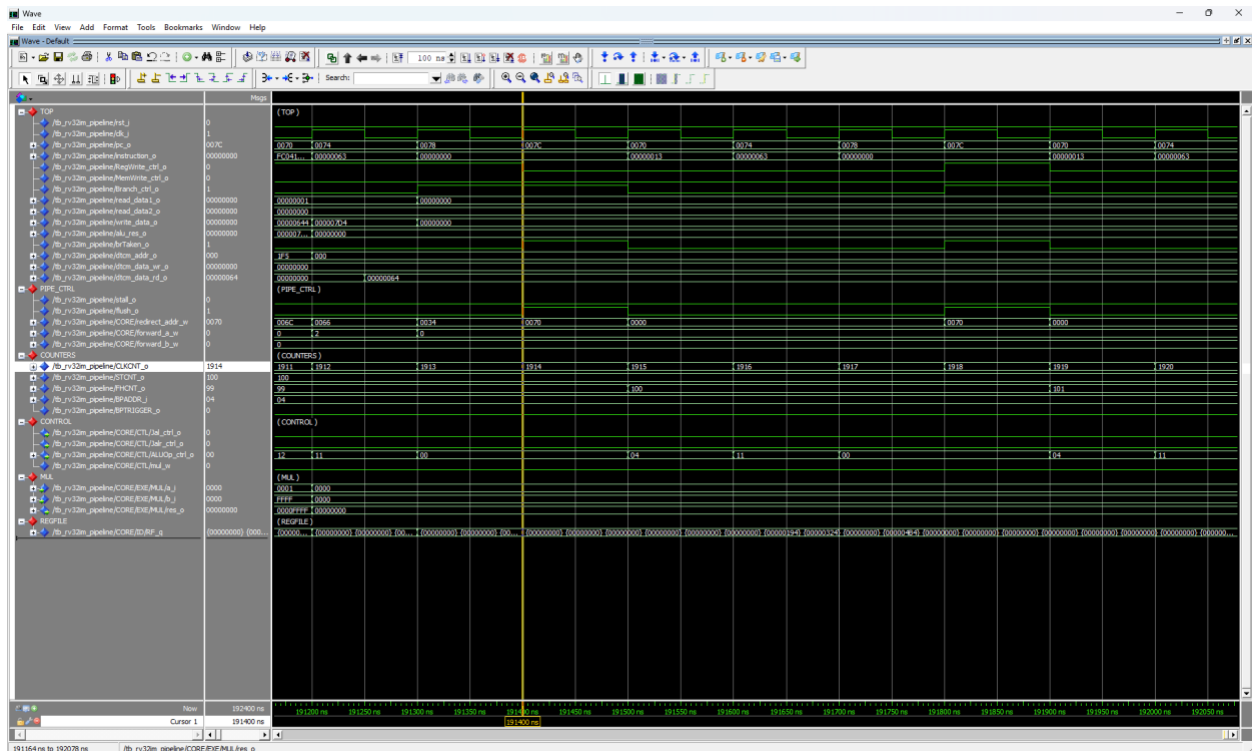
איור 29: test2 במימוש ה-single-cycle, מחזורים 31-40, עם רצף lw-sw-mul: כתובת ה-DTCM (dtcm_addr_o) היא תוצאת ה-ALU, והכתיבה מתבצעת באותו מחזור על השעון ההפוך



איור 30: test2 במימוש ה-single-cycle, מחזוריים 17-26, עם '1' = MemWrite_ctrl_o ופעולת הכופל



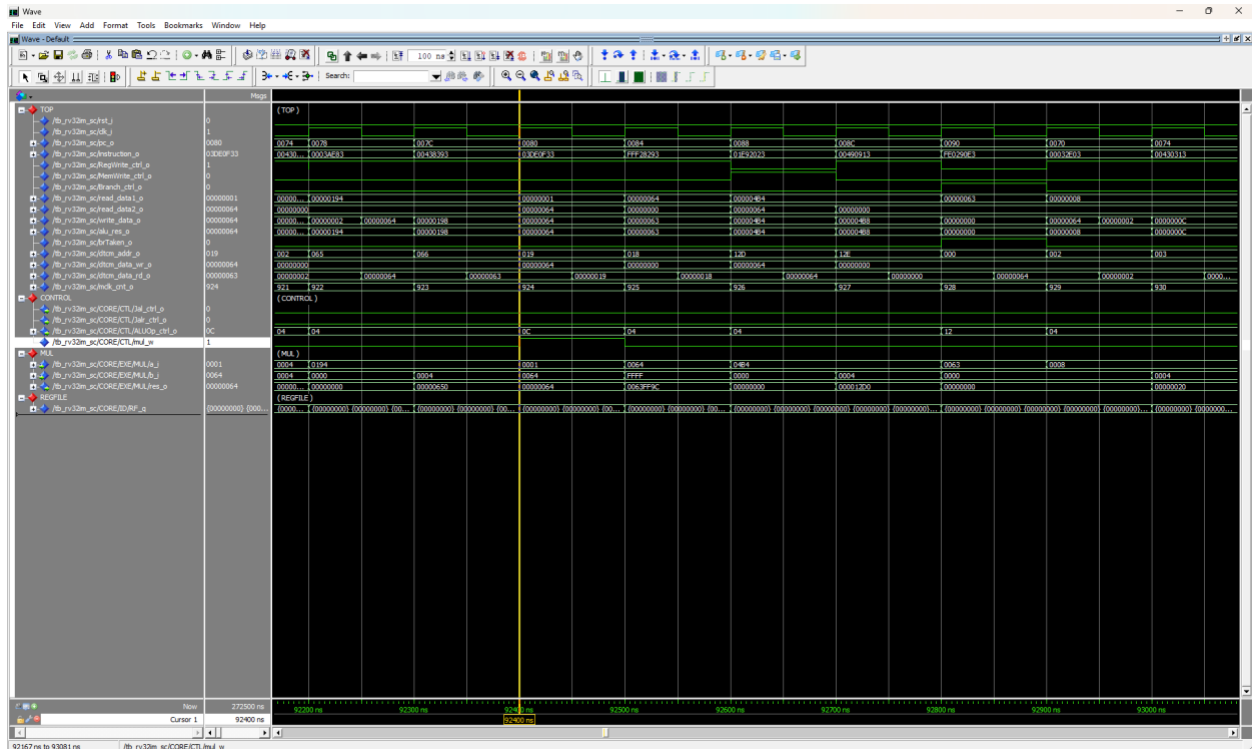
איור 31: test2 במימוש ה-pipeline, מחזורים 38–47 ($STCNT = 2$, $FHCNT = 1$), עם `forward_b_w` שמשתנה בין המקורות



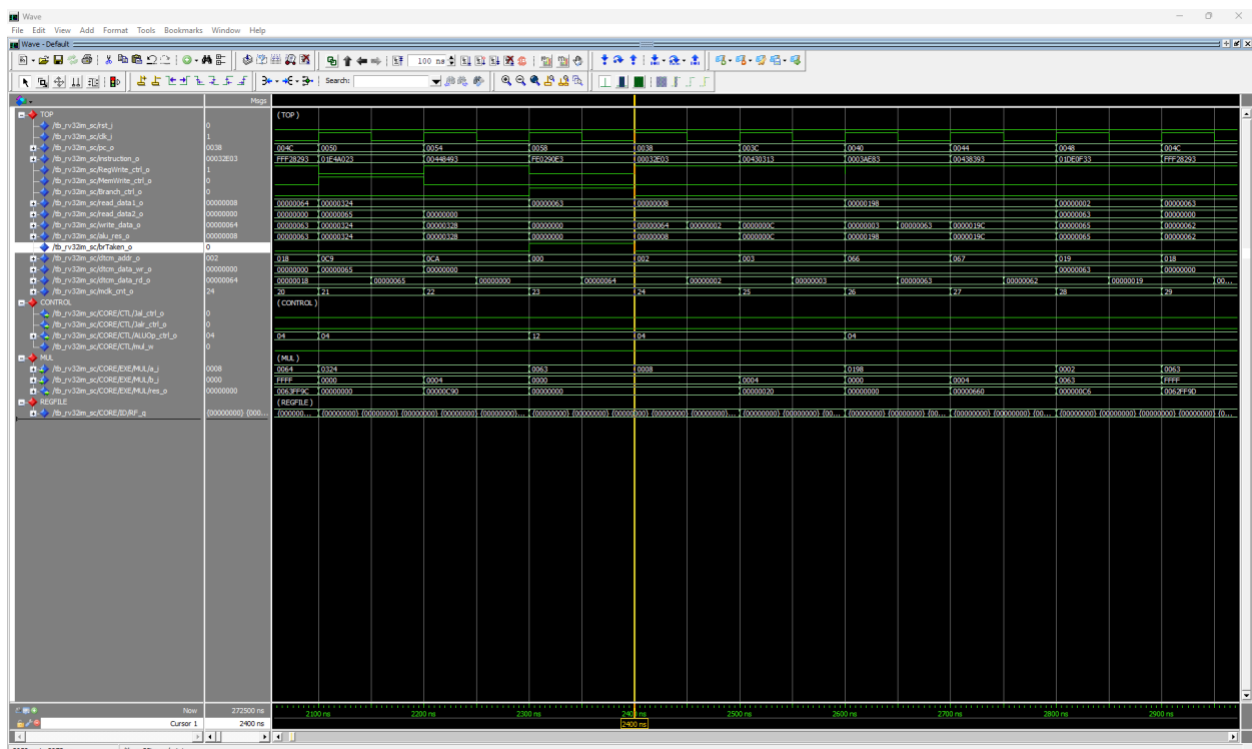
איור 32: test2 במימוש ה-pipeline, סיום התוכנית במחזור 1914. הערכים הסופיים הם $CLKCNT = 1914$, $STCNT = 100$, $FHCNT = 99$. מכאן ואילך המונים ממשיכים לעלות בגלל לולאת ה-`while(1)`, ולכן דגמנו אותם בדיוק ברגע זה

מבין ארבעת הטסטים, רק ב-`test2` נצפה מספר גדול של stalls ($STCNT = 100$), כלומר 100 מופעים של תבנית `1w` ואחריה מיד הוראה התלויה בתוצאתה. זהו המקרה שיחידת ה-`forwarding` אינה יכולה לכסות, ובדיאגרמה נראה שבכל אחד מהמופעים הללו `pc_o` נותר קפוא למחזור אחד.

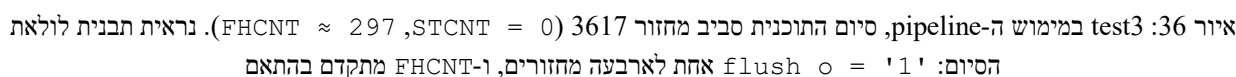
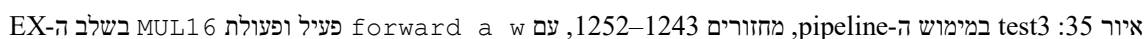
5.3.3 test3: לולאות והסתעפויות נלקחות



איור 33: test3 במימוש ה-single-cycle, מחזורים 921–930. נראית הסתעפות אחורה נלקחת: pc_o קופץ מ-0090 חזרה ל-0070 במחזור אחד, בלי עונש כלשהו

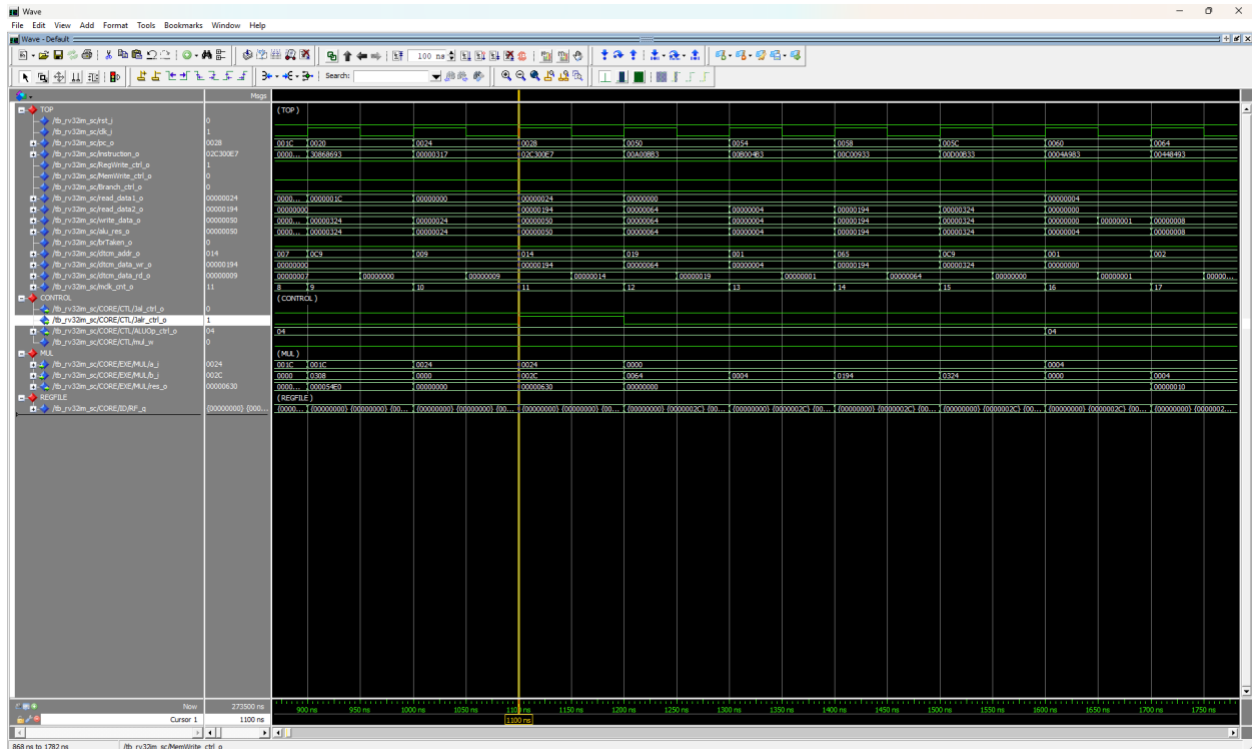


איור 34: test3 בממוש ה-single-cycle, מחזורים 20–29, הסתעפות אחורה מ-0058 ל-0038

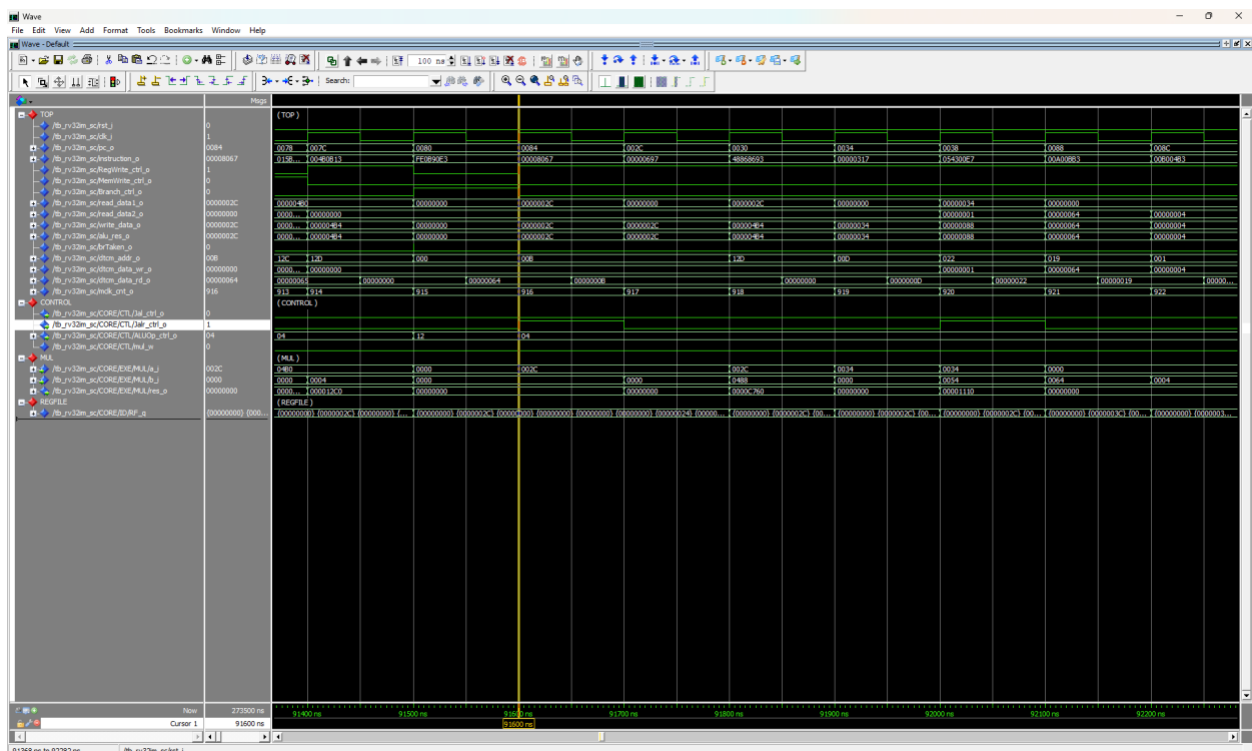


30

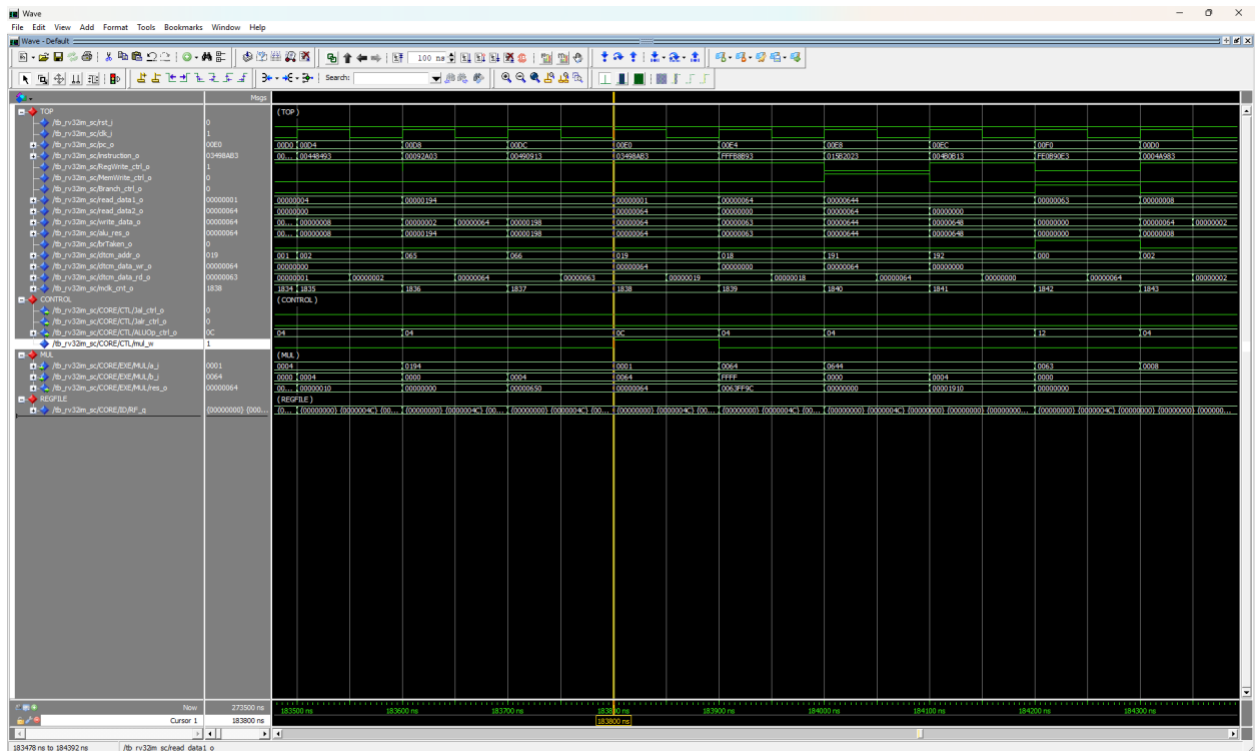
test4 5.3.4: קפיצות בלתי-מותנות וקריאות לפונקציות



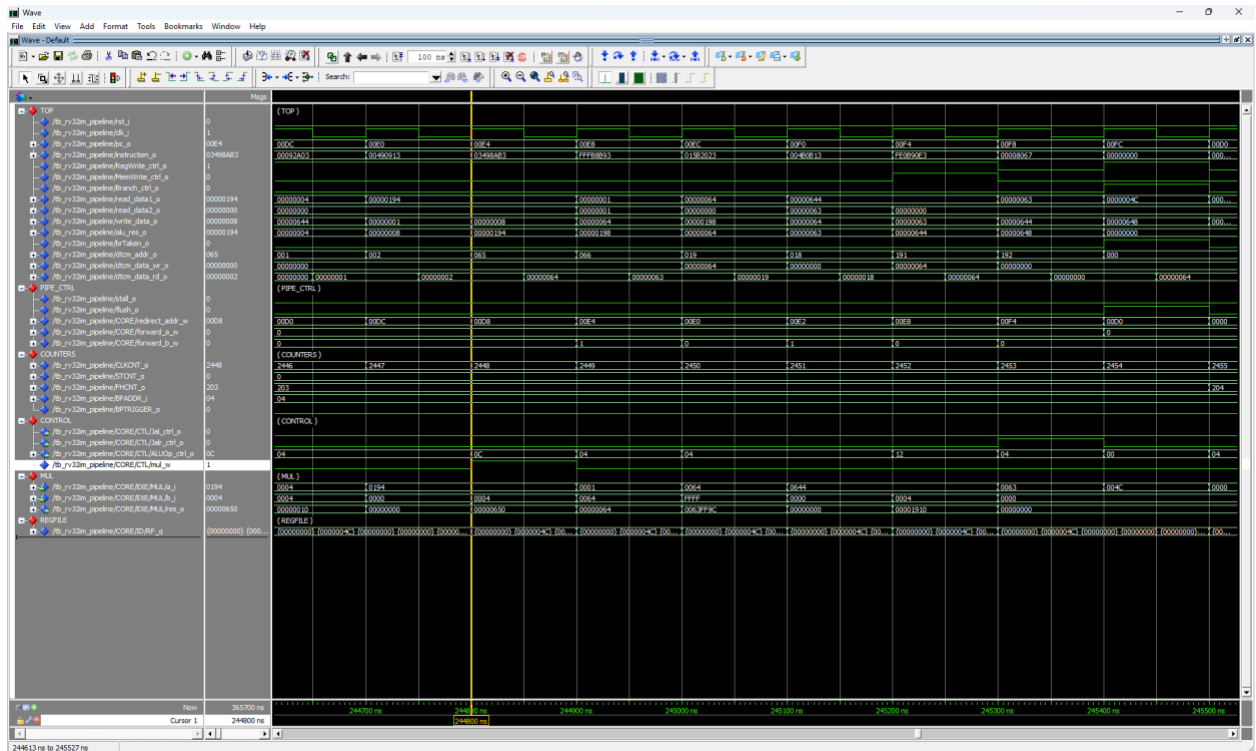
איור 37: test4 במימוש ה-single-cycle, מחזורי 8-17. האות '1' Jalr_ctrl_o = וה-PC קופץ מ-0028 ל-0050:
קריאה לפונקציה באמצעות jalr, שבה כתובת היעד מגיעה מתוצאת ה-ALU



איור 38: test4 במימוש ה-single-cycle, מחזוריים 913–922, עם jalr נוסף ופעולת כופל



איור 39: test4 במימוש ה-single-cycle, מחזורים 1834–1843, לולאה בטווח 00F0–00D0



איור 40: test4 במימוש ה-pipeline, מחזורים 2446–2455. לאורך כל הריצה `STCNT = 0`: קוד ה-benchmark אינו מכיל ולו תבנית load-use צמודה אחת, ולכן יחידת ה-interlock מעולם לא הופעלה

6. ניתוח PPA

שני התכנים קומפלו ב-Quartus Prime 21.1.0 Lite לרכיב EP4CE115F29C7, תחת אילוצי ה-sdc*. הבאים: שעון בן 50 MHz (20 ns) על clk_i, גזירה אוטומטית של שעון ה-PLL באמצעות derive_pll_clocks, ואיודאות שעון ריאליסטית באמצעות derive_clock_uncertainty.

6.1 שלוש טבלאות ה-PPA

שלוש טבלאות אפיון ה-PPA הנדרשות בסעיף 7 של הגדרת המשימה מרוכזות כאן יחד. צילומי דוחות ה-Quartus שמהם נלקחו הערכים, והניתוח המפורט של כל מדד, מופיעים בסעיפים 6.2–6.4.

Area						
	Logic elements	Registers	I/O pins	Embedded memory bits	Embedded 9-bit Multipliers	PLLs
Single-Cycle RV32IM	3,120	1,279	270	131,072	4	1
Pipelined RV32IM	3,384	1,696	297	131,072	4	1

טבלה 10: צריכת המשאבים של שני המימושים

צילומי דוח ה-Area של שני המימושים מצורפים באיורים 42 ו-43.

Performance			
	f_{max}	Critical path (what is the slowest submodule and why does it cause the critical path)	$f_{MCLK} (= f_{sysclk})$
Single-Cycle RV32IM	26.81 MHz	המסלול של sw/lw מה-IDCODE, דרך מרבכי ה-IDCODE וה-ALU, עד אוגר הכתובת של ה-DTCM. הכתובת היא תוצאת ה-ALU, ולכן כל השרשרת חייבת להסתיים לפני הגישה לזיכרון. השהיה 18.555 ns, 1.351 ns slack. הכתיבה ל-DTCM על השעון ההפוך הופכת אותו למסלול חצי-מחזור.	25 MHz
Pipelined RV32IM	41.84 MHz	שלב ה-ID: קריאה מקובץ אוגרים 32x32, פענוח, ייצור immediate ובדיקת load-use, הכל לתוך אוגר ה-ID/EX במחזור אחד. 41.84 MHz שקולים למסלול מחזור-מלא של 23.90 ns. המסלול מ-EX/MEM לכתובת ה-DTCM הוא 7.395 ns בלבד, ולכן אינו הקובע.	25 MHz

טבלה 11: תדר מרבי, מסלול קריטי ותדר העבודה בפועל

צילומי דוחות ה- f_{max} והמסלול הקריטי מצורפים באיורים 44 עד 47.

Power				
	Total power consumption	Static power consumption	Dynamic power consumption	I/O power consumption
Single-Cycle RV32IM	275.98 mW	102.39 mW	22.93 mW	150.67 mW
Pipelined RV32IM	299.10 mW	102.51 mW	31.29 mW	165.30 mW

טבלה 12: צריכת ההספק של שני המימושים

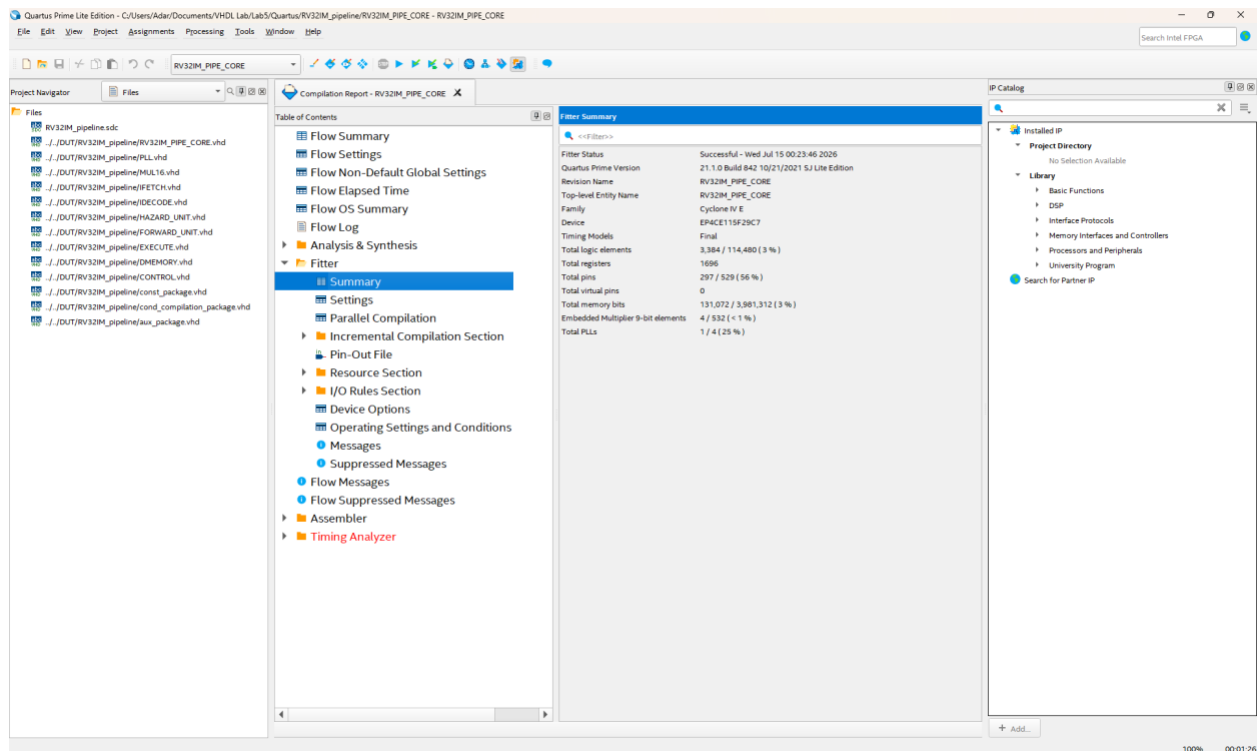
צילומי דוח ה-Power של שני המימושים מצורפים באיורים 48 ו-49.

ממד	Single-Cycle	pipeline	הפרש
Logic elements	3,120	3,384	264+ (%8.5+)
Registers	1,279	1,696	417+ (%32.6+)
I/O pins	270	297	27+ (%10.0+)
$f_{\max}$	26.81 MHz	41.84 MHz	56.1+ %
Total power	275.98 mW	299.10 mW	+23.12 mW (+8.4%)
Dynamic power	22.93 mW	31.29 mW	+8.36 mW (+36.5%)
I/O power	150.67 mW	165.30 mW	+14.63 mW (+9.7%)
Static power	102.39 mW	102.51 mW	+0.12 mW (+0.1%)

טבלה 13: סיכום ההפרשים בין שני המימושים בשלושת מדדי ה-PPA

Area 6.2: צריכת משאבים

איור 42: דוח Fitter Summary של RV32IM_CORE במימוש ה-single-cycle



איור 43: דוח Fitter Summary של RV32IM_PIPE_CORE במימוש ה-pipeline

ניתוח

- מספר האוגרים גדל ב-32.6%, הגידול הגדול ביותר מבין המדדים, וזהו המחיר הישיר של ה-pipeline. ארבעה אוגרי pipeline (IF/ID, EX/MEM, MEM/WB, ID/EX) חייבים לשמור בכל מחזור את ההוראה, ה-PC, שני האופרנדים, ה-immediate, מזהי האוגרים וכל אותות הבקרה. נוספים להם שלושת מוני הניפוי ואוגר .BPADDR.
- הלוגיקה גדלה הרבה פחות, ב-8.5% בלבד. מסלול הנתונים עצמו לא הוכפל; נוספו רק FORWARD_UNIT, HAZARD_UNIT, מרבבי ה-forwarding ולוגיקת ה-redirect. הפער בין +32.6% באוגרים ל-8.5% בלוגיקה מאפיין pipeline: היא קונה זמן בעזרת אוגרים, לא בעזרת לוגיקה.
- הזיכרון המשובץ זהה בשני המימושים: 131,072 ביט, כלומר $2 \times 8 \text{ KiB}$ של ה-ITCM וה-DTCM. ה-pipeline אינו משנה את מבנה הזיכרון אלא רק את אופן הגישה אליו.
- בשני המימושים נוצלו 4 כופלים משובצים, ומכאן ש-MUL16 מופה לארבעה כופלים חומריים כנדרש באלגוריתם שבסעיף 3.2.1, ולא סונתה ללוגיקה.
- הגידול של +27 פנינים נובע מאותות הניפוי החדשים: stall_o, flush_o, BPTRIGGER_o, FHCNT_o[7..0], STCNT_o[7..0], CLKCNT_o[15..0], BPADDR_i[7..0].

Performance 6.3: תדר מרבי ומסלול קריטי

The screenshot shows the Timing Analyzer window for the project 'RV32IM_CORE'. The 'Set Operating Conditions' panel on the left has 'Slow 1200mV BSC Model' selected. The 'Tasks' panel shows a list of tasks, with 'Report Fmax Summary' highlighted. The 'Report' panel shows the 'Timing Analyzer Summary' with 'Fmax Summary' expanded. The main panel displays the 'Slow 1200mV BSC Model' results table.

	Fmax	Restricted Fmax	Clock Name	Note
1	26.81 MHz	26.81 MHz	\GD\MCLK\altpll_component\pllclk[0]	
2	73.62 MHz	73.62 MHz	alters_reserved_tck	

This panel reports FMAX for every clock in the design, regardless of the user-specified clock periods. FMAX is only computed for paths where the source and destination registers or ports are driven by the same clock. Paths of different clocks, including generated clocks, are ignored. For paths between a clock and its inversion, FMAX is computed as if the rising and falling edges are scaled along with FMAX, such that the duty cycle (in terms of a percentage) is maintained. Altera recommends that you always use clock constraints and other slack reports for sign-off analysis.

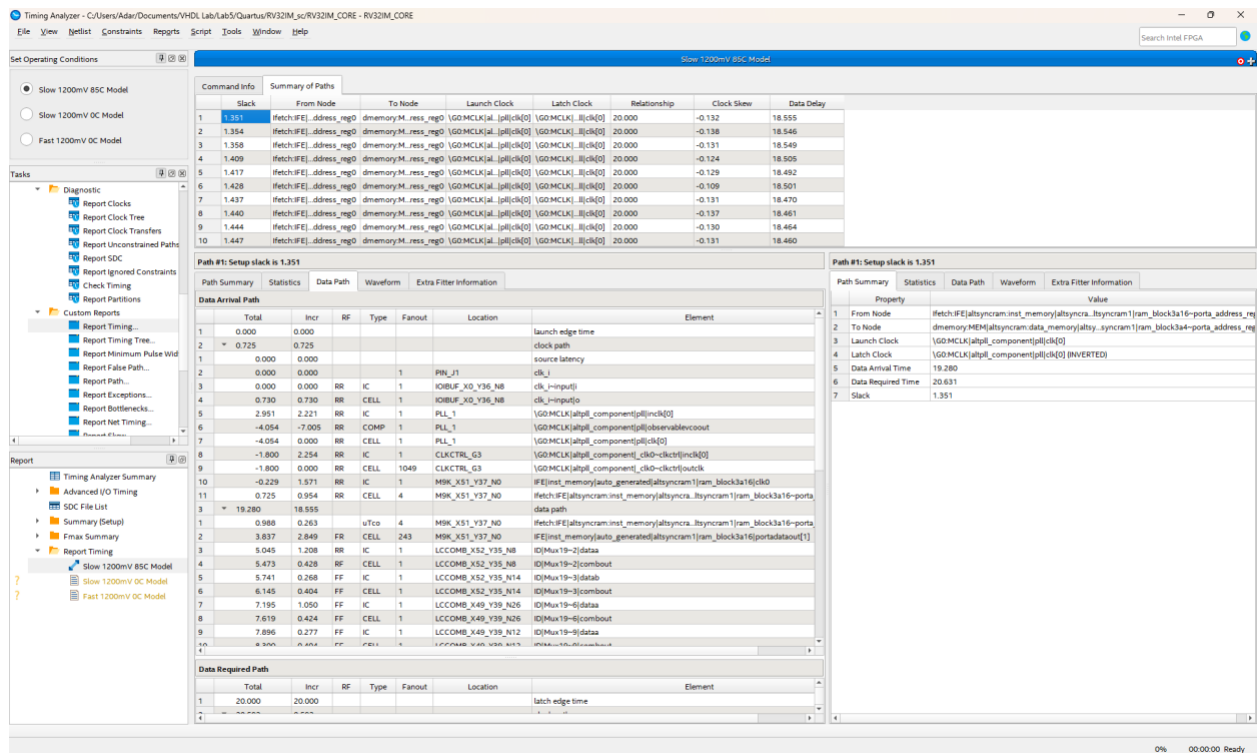
איור 44: דוח Fmax Summary של מימוש ה-single-cycle: 26.81 MHz על שעון הליבה MCLK. השורה השנייה (alters_reserved_tck) היא שעון ה-JTAG ואינה חלק מהתכן

The screenshot shows the Timing Analyzer window for the project 'RV32IM_PIPE_CORE'. The 'Set Operating Conditions' panel on the left has 'Slow 1200mV BSC Model' selected. The 'Tasks' panel shows a list of tasks, with 'Report Fmax Summary' highlighted. The 'Report' panel shows the 'Timing Analyzer Summary' with 'Fmax Summary' expanded. The main panel displays the 'Slow 1200mV BSC Model' results table.

	Fmax	Restricted Fmax	Clock Name	Note
1	41.84 MHz	41.84 MHz	\GD\MCLK\altpll_component\pllclk[0]	
2	88.26 MHz	88.26 MHz	alters_reserved_tck	

This panel reports FMAX for every clock in the design, regardless of the user-specified clock periods. FMAX is only computed for paths where the source and destination registers or ports are driven by the same clock. Paths of different clocks, including generated clocks, are ignored. For paths between a clock and its inversion, FMAX is computed as if the rising and falling edges are scaled along with FMAX, such that the duty cycle (in terms of a percentage) is maintained. Altera recommends that you always use clock constraints and other slack reports for sign-off analysis.

איור 45: דוח Fmax Summary של מימוש ה-pipeline: 41.84 MHz על שעון הליבה MCLK



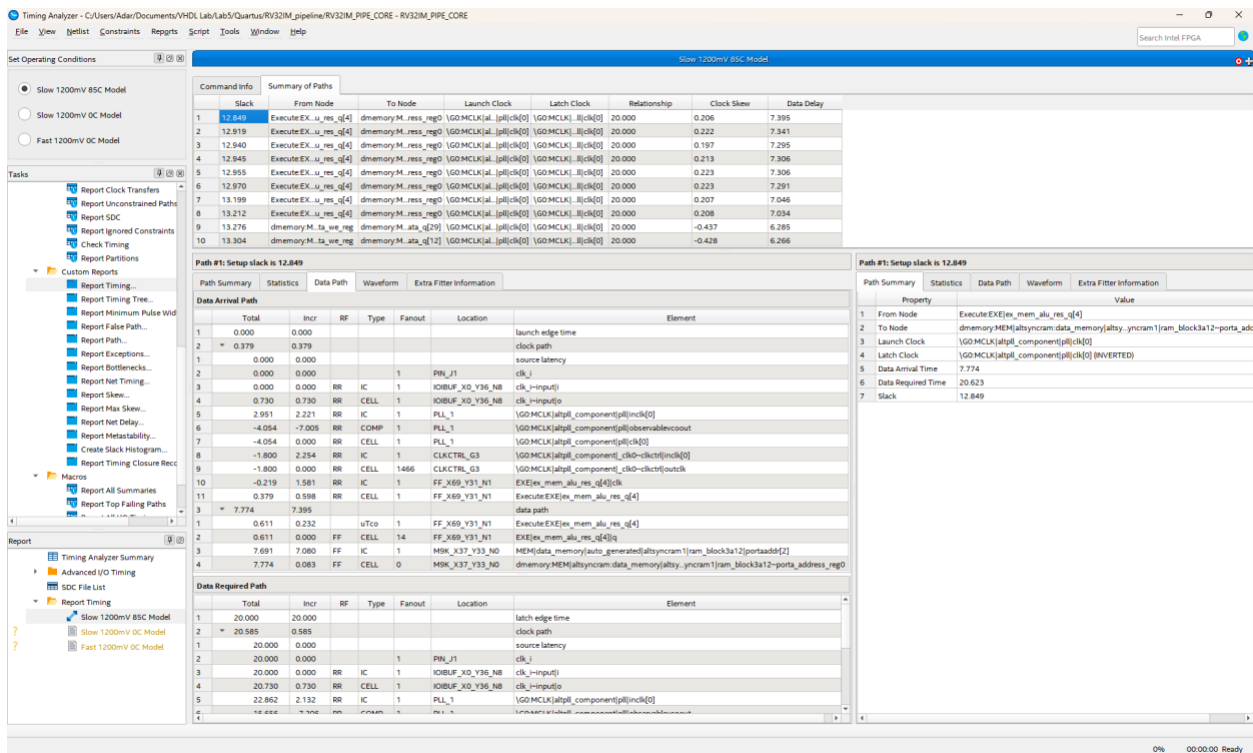
איור 46: המסלול הקריטי של מיושם ה-`single-cycle`, מ-`ram_block|porta_address_reg` אל `dmemory:MEM|...|porta_address_reg` מסומן `INVERTED`, ולכן זהו מסלול חצי-מחזור

חישוב ה- $f_{\max}$ מתוך המסלול

השהיית המסלול היא 18.555 ns בלבד, ולכאורה הייתה מאפשרת 53 MHz. אלא שכפי שהוסבר בסעיף 3.1, ה-DTCM נכתב על השעון ההפוך, וזה נראה מפורשות בשדה `Latch Clock` שבדוח, המסומן `INVERTED`. לכן המסלול כולו חייב להסתיים בתוך חצי מחזור שעון:

$$f_{\max} = 1 / (2 \cdot t_{\text{path}}) = 1 / (2 \cdot 18.65 \text{ ns}) \approx 26.8 \text{ MHz}$$

החישוב מתאים ל-26.81 MHz שדיווח ה-Timing Analyzer, ומכאן שזיהינו נכון את המסלול.



איור 47: דוח Report Timing של ה-pipeline: המסלולים מ-Execute:EXE|ex_mem_alu_res_q אל אוגר הכתובת של DTCM-ה, עם slack חיובי של 12.849 ns

זיהוי המסלול הקריטי של ה-pipeline

ב-pipeline, בניגוד למימוש ה-single-cycle, המסלול בעל ה-slack הקטן ביותר אינו בהכרח המסלול שקובע את f_{max} , משום שבתכן מתקיימים בריזמנית מסלולי חצי-מחזור (כל מה שנכנס לאוגרי ה-DTCM הנדגמים על השעון ההפוך) ומסלולי מחזור-מלא (בין אוגרי ה-pipeline). דרישת הזמן של הראשונים משתנה כמו $T/2$ ושל האחרונים כמו T , ולכן ה-slack אינו מדד השוואה ישיר ביניהם. החישוב שבטבלה הבאה ממחיש זאת:

סוג המסלול	השהיה	תדר מרבי שהמסלול מתיר
EX/MEM → כתובת ה-DTCM (חצי-מחזור, מהדוח)	7.774 ns	$1 / (2 \cdot 7.774) \approx 64 \text{ MHz}$
המסלול הקובע בפועל (מחזור-מלא)	23.90 ns	$1 / 23.90 = 41.84 \text{ MHz}$

טבלה 14: השהיית מסלול חצי-מחזור ומסלול המחזור-המלא ב-pipeline והתדר שכל אחד מהם מתיר; מכאן שהמסלול בעל ה-slack הקטן ביותר אינו זה שקובע את f_{max}

מכאן שהמסלול שקובע את 41.84 MHz הוא מסלול מחזור-מלא באורך $\approx 23.9 \text{ ns}$, ולא מסלול הגישה ל-DTCM. מבין חמשת השלבים, השלב היחיד שמצטבר בו עומק לוגי כזה הוא שלב ה-ID. אותה הוראה עצמה מפעילה בריזמנית את קריאת קובץ האוגרים 32×32 (עץ מרבבים רחב הממומש בלוגיקה ולא בזיכרון משובץ), את הפענוח המלא ב-CONTROL, את בחירת אחת מחמש תבניות ה-immediate והרחבת הסימן, ואת בדיקת ה-load-use ב-HAZARD_UNIT, וכל אלה חייבים להתייצב לתוך אוגר ה-ID/EX בתוך מחזור אחד. לשם השוואה, שלב ה-EX אמנם מכיל את ה-ALU ואת MUL16, אך MUL16 ממומש על כופלים חומרתיים מהירים ולא על לוגיקה, ושלבי IF ו-MEM נשלטים על ידי בלוקי ה-M9K שהשהייתם נמוכה בהרבה.

השוואה לערך התיאורטי

לפי המשימה, תכן pipeline איכותי אמור להתקרב ל-

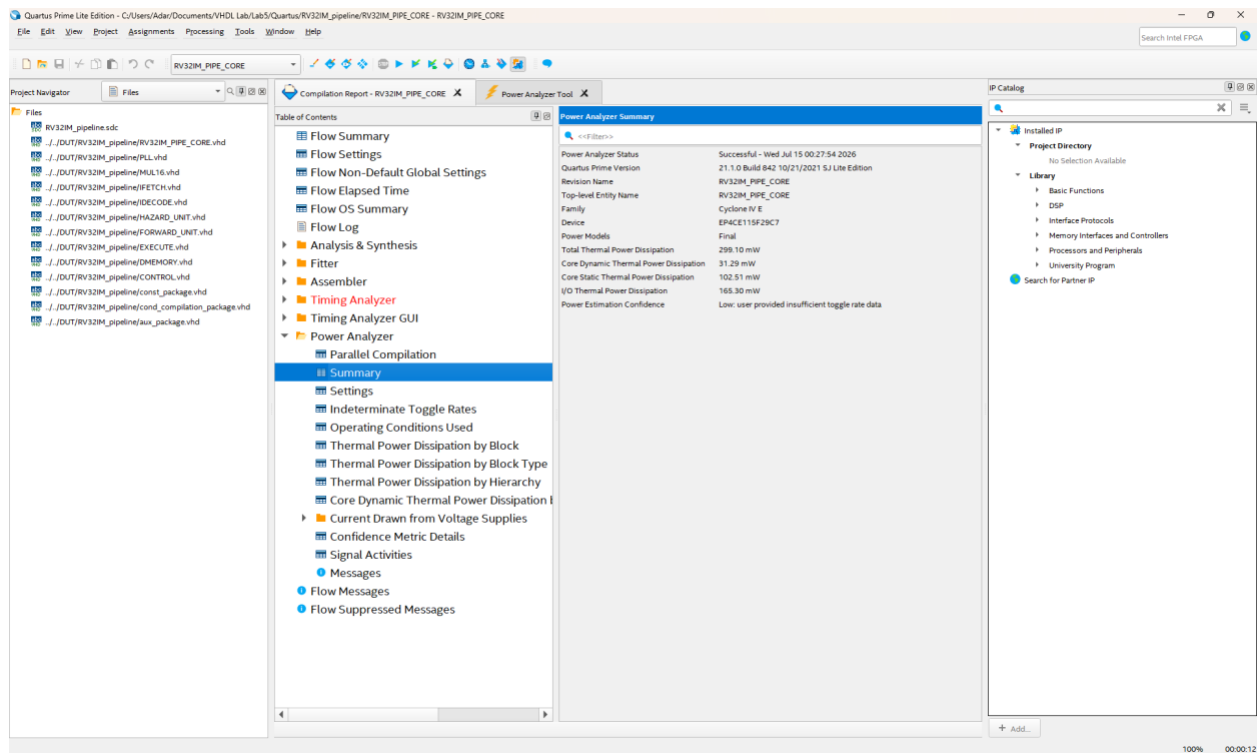
$$f_{\max}^{\text{pipeline}} = 5 \cdot f_{\max}^{\text{single-cycle}} = 5 \cdot 26.81 = 134.05 \text{ MHz}$$

בפועל התקבלו 41.84 MHz, כלומר 31% מהערך התיאורטי ושיפור של $1.56 \times$ בלבד ביחס ל-single-cycle. הפער נובע מארבע סיבות מבניות:

1. ההנחה שמאחורי הכפולה $5 \times$ היא שניתן לחלק את המסלול לחמישה חלקים שווים. בפועל, השהיית הגישה לבלוקי ה-M9K היא רכיב קשיח שאינו ניתן לחיתוך על ידי הוספת אוגרים, והיא מהווה רצפת השהיה ששלבי ה-IF וה-MEM אינם יכולים לרדת מתחתיה.
2. הכתיבה על השעון ההפוך נשמרה גם ב-pipeline, ולכן המסלול מאוגר ה-EX/MEM אל כתובת ה-DTCM נותר מסלול חצי-מחזור. הוא לבדו כבר מגביל את התכן ל- ~ 64 MHz, הרבה מתחת ל-134 MHz התיאורטיים.
3. השלבים אינם מאוזנים ביניהם. שלב ה-ID מרכז קריאה מקובץ אוגרים 32×32 , פענוח מלא, ייצור חמש תבניות immediate ובדיקת hazards; שלב ה-EX מוסיף מרבבי forwarding לפני ה-ALU ומכיל גם את MUL16. השלב האיטי הוא שקובע את התדר.
4. ל-pipeline עצמו יש תקורה: כל שלב משלם clock-to-Q של אוגר ה-pipeline ו-setup של האוגר הבא, ובנוסף אי-ודאות השעון. תקורה זו אינה מתחלקת ב-5.

Power 6.4: צריכת הספק

איור 48: דוח Power Analyzer Summary של מימוש ה-single-cycle



איור 49: דוח Power Analyzer Summary של מימוש ה-pipeline

ניתוח

- ההספק הסטטי כמעט זהה בשני המימושים (102.39 מול 102.51 mW). זהו ההספק הזליגה של הרכיב כולו, והוא תלוי בעיקר בגודל הרכיב ובטמפרטורה ולא במידת ניצולו; שני התכנים מנצלים רק 3% מהלוגיקה.
 - ההספק הדינמי גדל ב-36.5%, הגידול היחסי הגדול ביותר מבין מדדי ההספק. הוא נובע ישירות מ-417 האוגרים: כל אוגר pipeline מחליף מצב בכל מחזור שעון, וההספק הדינמי פרופורציוני למספר המעברים.
 - הספק ה-I/O הוא המרכיב הדומיננטי, למעלה מ-55% מסך ההספק בשני המימושים. התכן מוציא החוצה מספר רב מאוד של אותות ניפוי ברוחב מלא (270 ו-297 פינים בהתאמה), והנעת פין חיצוני יקרה בהרבה מהחלפת מצב של אוגר פנימי. בתכן ייצור אמיתי אותות אלו לא היו מוצאים החוצה כלל, וההספק היה נשלט על ידי הרכיב הדינמי.
- שני הדוחות מציינים Power Estimation Confidence: Low - user provided insufficient toggle rate data. כלומר האומדן מבוסס על הנחות ברירת מחדל של קצבי החלפה ולא על קובץ פעילות (* .vcd) מהסימולציה. לפיכך הערכים המוחלטים אינם מדויקים, אך ההשוואה היחסית בין שני המימושים נעשתה תחת אותן הנחות בדיוק ולכן היא תקפה.

7. ניתוח IPC על בסיס אפליקציות benchmark

סעיף 8.c.iii של המשימה דורש לאמת את ה-IPC בהשוואה בין שני אגפי המשוואה שבסעיף 4.5. אגף אחד מחושב ממוני החומרה, והאגף השני הוא מספר ההוראות שהאפליקציה מבצעת בפועל. כדי שההשוואה תהיה משמעותית, מספר ההוראות הזה צריך להיות ידוע במדויק.

7.1 מקור בלתי-תלוי למספר ההוראות

לשם כך נעזרנו במאפיין שתואר בסעיף 3.3. במימוש ה-single-cycle מתקיים $IPC = 1$ בהגדרה, ולכן מונה מחזורי השעון שלו בסוף הריצה שווה בדיוק למספר ההוראות שהאפליקציה מבצעת. שני המימושים מריצים את אותו קוד בינארי ומייצרים DTCM זהה (סעיף 5.2), ולכן מונה המחזוריים שלו משמש לנו מודל זהב לבדיקת מונה ההוראות של ה-pipeline.

IPC	הוראות שפרשו $CLKCNT - (STCNT + 4 + 3 \cdot FHCNT)$	FHCNT	STCNT	CLKCNT	מחזורי single-cycle (= מספר ההוראות)	benchmark
0.807	134	6	10	166	134	test1
0.7905	1,513	99	100	1,914	1,514	test2
0.753≈	2,722≈	297≈	0	3,617≈	2,725	test3
0.749≈	2,731≈	303≈	0	3,644≈	2,735	test4

טבלה 15: מוני החומרה וחישוב ה-IPC לאפליקציות ה-benchmark. ערכי test3 ו-test4 נקראו מדיאגרמות הגלים של סיום התוכנית (איורים 36 ו-41) ולכן מסומנים ב-≈

אימות המשוואה

עבור test2, שני אגפי המשוואה מתלכדים כמעט במלואם:

$$1,914 - (100 + 4 + 3 \cdot 99) = 1,914 - 401 = 1,513 \text{ versus } 1,514$$

הפער הוא הוראה אחת בלבד (0.07%), והוא נובע מרגע הדגימה של המונים. המונים נקראים ברגע שמזוהה אירוע ה-flush של הקפיצה העצמית, בעוד ההוראות האחרונות עדיין נמצאות ב-pipeline. גם עבור test3 ו-test4 מתקבלת התאמה טובה: $2,722 \approx$ מול $2,725$, ו- $2,731 \approx$ מול $2,735$. בהתאמה. ההתאמה מאמתת את נכונות המונים, וגם את הקבועים 4 (מילוי ה-pipeline) ו-3 (עומק ה-flush) שבנוסחה.

7.2 זמן ריצה בפועל

ערך IPC קטן מ-1 אינו מעיד שה-pipeline נחותה; הוא מתאר את יעילות המחזוריים בלבד. זמן הריצה בפועל הוא מכפלה של מספר המחזוריים בתקופת השעון:

$$T_{\text{exec}} = \text{Cycles} \times T_{\text{clk}} = \text{Cycles} / f$$

שיפור	pipeline: מחזוריים $\times 23.90 \text{ ns}$	SC: מחזוריים $\times 37.30 \text{ ns}$	benchmark
$1.23 \times$	$1,914 \rightarrow 45.75 \mu\text{s}$	$1,514 \rightarrow 56.47 \mu\text{s}$	test2
$1.18 \times$	$\approx 3,617 \rightarrow 86.45 \mu\text{s}$	$2,725 \rightarrow 101.64 \mu\text{s}$	test3
$1.17 \times$	$\approx 3,644 \rightarrow 87.09 \mu\text{s}$	$2,735 \rightarrow 102.02 \mu\text{s}$	test4

טבלה 16: זמן ריצה בפועל כאשר כל מימוש פועל בתדר המרבי שלו (26.81 ו-41.84 MHz בהתאמה)

השיפור בפועל, $1.17 \times$ עד $1.23 \times$, תואם בדיוק את המכפלה של שני הגורמים: 1.56 (שיפור התדר) כפול 0.79 (ה-IPC) שווה 1.23 .

בתצורה הנוכחית, שני המימושים מוגדרים לאותו יחס PLL ($G_PLL_DIV = 2$, $G_PLL_MUL = 1$), ולכן שניהם עובדים ב- 25 MHz . בתדר זה זמן הריצה נקבע לפי מספר המחזורים בלבד, ולכן ה-pipeline יוצא דווקא איטי יותר: ב-test2 מתקבלים $76.56 \mu\text{s}$ מול $60.56 \mu\text{s}$.

כדי לתקן זאת יש להעלות את יחס ה-PLL של ה-pipeline כך שינצל את 41.84 MHz שהוא מסוגל לו (למשל 40 MHz באמצעות $G_PLL_DIV = 5$, $G_PLL_MUL = 4$). ללא שינוי זה, העבודה שהושקעה בתכנון ה-pipeline אינה מתורגמת לשיפור ביצועים בפועל.

8. סיכום ומסקנות

בעבודה זו תכננו שני מימושים של מעבד RV32IM תואם RISC-V, סינתזנו אותם ואימתנו את פעולתם. המימוש הראשון הוא single-cycle, והשני pipeline סקלרי בן חמישה שלבים עם forwarding מלא ו-interlock ל-load--use.

תוצאות עיקריות

- ארבע אפליקציות ה-benchmark הופקו בשני המימושים, ותמונות ה-DTCM שהתקבלו זהות למודל הזהב של RARS, וגם זהות בית-בית זו לזו. שני המעבדים נכונים אפוא מבחינה פונקציונלית.
- הכופל מופה כמצופה: ארבעה כופלים משובצים בני 9 ביט נוצלו בפועל, בהתאם לאלגוריתם הנדרש.
- נוסחת ה-IPC אומתה: מונה ההוראות המחושב מהמונים תואם את מספר ההוראות שנמדד במימוש ה-single-cycle בדיוק של הוראה אחת.

מסקנות PPA

- ה-pipeline קונה ביצועים באגרים: +32.6% אוגרים לעומת +8.5% לוגיקה בלבד. כך נראית המרה של השהיה בתפוקה.
- מחיר ההספק (+8.4% סך הכול, +36.5% בהספק הדינמי) קטן ביחס לשיפור התדר (+56%), ולכן יחס הביצועים-להספק השתפר.
- הפער מהיעד התיאורטי (41.84 מול 134 MHz) נובע בעיקרו מרצפת ההשהיה של הזיכרון המשובץ. תורמות לו גם שמירת הכתיבה על השעון ההפוך וחוסר האיזון בין השלבים.

שיפורים אפשריים

1. העלאת יחס ה-PLL של ה-pipeline. זה השינוי הזול ביותר לביצוע והתועלת ממנו הגדולה ביותר, וכל עוד לא נעשה ה-pipeline פועל ב-25 MHz ואינו מנצל את יתרונה.
2. הקדמת הכרעת ההסתעפויות משלב MEM לשלב EX או ID הייתה מקטינה את עומק ה-flush מ-3 ל-2 או ל-1. ב-test4, שבו נספרו כ-303 אירועי flush, החיסכון היה מגיע לכ-300 עד 600 מחזורים.
3. איזון עומק הלוגיקה בין השלבים. פיצול שלב ה-ID העמוס היה מעלה את f_{max} ומקרב אותו לערך התיאורטי.
4. צמצום אותות הניפוי בתכן הסופי. הספק ה-I/O מהווה למעלה מ-55% מסך ההספק, ורובו נובע מפנינים שנועדו לניפוי בלבד.